

Resumos Consolidados - Blockchain e Web 3.0

Este documento unifica os resumos de estudo das aulas 1 a 14.

Aula 1

Disciplina: Blockchain e Web 3.0 **Vídeo de Referência:** @[aulas/videos/Aula 1 - Introdução à Blockchain (Parte 1).mp4]

🔑 1. Visão Geral e Objetivos

- **Contexto:** Esta aula inicial apresenta os conceitos fundamentais de Blockchain e Web 3.0, contrapondo a evolução da internet (Web 1.0 e 2.0) com as novas tecnologias descentralizadas. A importância reside em entender as limitações dos sistemas centralizados atuais e como a blockchain propõe soluções inovadoras de segurança, autonomia e distribuição de confiança.
- **Objetivos de Aprendizagem:** Ao final da aula, o aluno deve compreender a diferença arquitetural entre redes centralizadas e descentralizadas, o papel fundamental das funções hash na imutabilidade dos dados, a necessidade do consenso em redes distribuídas para evitar fraudes, e a utilidade dos smart contracts.

🔑 2. Conceitos-Chave e Definições

- **Web 1.0:** Primeira fase da internet, caracterizada por páginas estáticas (apenas leitura) onde os usuários eram puramente consumidores de conteúdo, sem interação dinâmica.
- **Web 2.0:** A internet interativa e social, onde os usuários produzem conteúdo (ex: redes sociais, fóruns), porém baseada em plataformas e servidores **centralizados**, controlados por grandes empresas corporativas.
- **Web 3.0 (Web3):** A nova geração da internet focada na **descentralização**, utilizando tecnologias como blockchain para que os usuários tenham posse e controle de seus próprios dados, transacionando sem a necessidade de um intermediário central.
- **Descentralização:** Arquitetura de rede (como a P2P - *Peer-to-Peer*) onde não há um servidor central único; o controle, as validações e o armazenamento de dados são distribuídos entre os vários "nós" (computadores) participantes da rede.
- **Hash:** Uma função matemática e criptográfica que transforma qualquer dado de entrada em uma sequência de caracteres de tamanho fixo (ex: algoritmo SHA-512). É irreversível e qualquer mínima alteração na entrada gera um hash completamente diferente (propriedade de efeito avalanche).
- **Gasto Duplo (Double Spending):** O problema clássico da computação de se utilizar o mesmo ativo digital (como um arquivo ou moeda virtual) mais de uma vez. A blockchain resolve isso usando o consenso e um livro-razão público.
- **Consenso:** Mecanismo através do qual os nós de uma rede descentralizada concordam coletivamente sobre qual é a versão verdadeira da realidade (ex: quais transações são válidas), garantindo a segurança da rede sem depender de um servidor central confiável.
- **Smart Contracts (Contratos Inteligentes):** Programas de computador autoexecutáveis que rodam dentro de uma blockchain, executando lógicas e regras automaticamente quando

condições pré-definidas são atendidas, dispensando intermediários humanos ou institucionais.

- **DApps (Decentralized Applications):** Aplicativos descentralizados que funcionam conectando um front-end tradicional a um back-end rodando em uma blockchain (usando smart contracts), em vez de servidores centralizados.

3. Explicação Detalhada por Tópicos

- **A Evolução da Web e a Arquitetura de Rede:**

- Na **Web 1.0**, o modelo era majoritariamente estático.
- Com a **Web 2.0**, o conteúdo passou a ser dinâmico. Tecnologias como AJAX e frameworks de front/back-end permitiram interações ricas, mas solidificaram a arquitetura cliente-servidor (centralizada).
- **O problema da centralização:** Se o servidor central cai ou toma decisões unilaterais (como censura ou venda de dados), os usuários são afetados sem recurso. Há também pontos únicos de falha (*Single Points of Failure*) para ataques hackers.

- **A Transição para a Web 3.0:**

- A Web3 adota arquiteturas *Peer-to-Peer* (P2P), onde todos os nós podem agir como clientes e servidores simultaneamente, inspirando-se em tecnologias mais antigas como o Torrent.
- O grande salto da Web3 foi adicionar uma camada de **registro de estado unificado e seguro**, permitindo que uma rede P2P mantenha um histórico imutável de transações.

- **A Solução da Blockchain e a Matemática do Hash:**

- Em uma rede descentralizada, precisamos de garantias criptográficas em vez de confiança institucional.
- Cada bloco de informações na blockchain contém o **hash do bloco anterior**. Isso cria uma corrente inquebrável.
- Se um nó malicioso tentar alterar uma transação no passado, o hash daquele bloco mudará. Como o bloco seguinte aponta para o hash antigo, a corrente se quebra, e a rede imediatamente rejeita a fraude.

- **O Problema do Gasto Duplo e o Consenso:**

- No mundo digital tradicional, copiar algo é perfeitamente exato. Isso impede a escassez digital necessária para o dinheiro.
- Para que uma transação (como o envio de uma moeda) seja validada, os nós precisam chegar a um **consenso** (geralmente mais de 50% de aprovação) de que os fundos existem e não foram gastos duplamente.
- Os validadores (como mineradores) investem recursos computacionais ou financeiros para participar da rede e são recompensados financeiramente se agirem de forma honesta, criando um forte **incentivo econômico** contra fraudes.

- **Expandindo o Conceito: Smart Contracts e DApps:**

- Inicialmente, a blockchain servia apenas como um "livro-razão" de saldos (como no Bitcoin).
- Redes mais recentes (ex: Ethereum) introduziram os **Smart Contracts**, permitindo salvar não só dados, mas **lógica de programação** imutável na rede.
- Isso permitiu a criação das **DApps**, abrindo portas para sistemas financeiros complexos, jogos e organizações descentralizadas rodando integralmente na Web3.

4. Exemplos Práticos e Analogias

- **Analogia da Foto vs. Dinheiro:** O professor usa a foto do celular para ilustrar o "gasto duplo". Se eu tiro uma foto e envio no WhatsApp, eu mantenho a minha cópia e você recebe outra. Essa "cópia infinita" é ótima para informações, mas destrutiva para sistemas financeiros. Se eu envio uma nota de R\$ 10, eu não posso continuar com ela. A blockchain funciona como o mecanismo que garante que o "arquivo do dinheiro" trocou de mãos e não foi simplesmente copiado.
- **Analogia do Livro-Razão Público:** Em vez de confiar no servidor de um banco (que pode ser hackeado internamente ou falhar), imagine que todos em uma sala possuem uma cópia exata de um livro de contabilidade. Se alguém tentar dizer "eu tenho 1 milhão", a sala inteira olha para seus livros, percebe a mentira e descarta a transação. Para ter sucesso, o falsificador teria que invadir e alterar o livro de mais da metade da sala ao mesmo tempo (o que se torna inviável computacionalmente em uma rede global).

5. Resumo dos Pontos Críticos (O que vai cair na prova)

- A diferença entre **Web 2.0** (interativa e centralizada) e **Web 3.0** (focada em posse, descentralização e redes P2P).
- A **função Hash** é unidirecional, gera tamanhos fixos e tem o "efeito avalanche". É a base técnica que garante a imutabilidade dos blocos na "corrente".
- O **Gasto Duplo (Double Spending)** é o problema de duplicação digital que a blockchain resolve através do registro público e dos mecanismos de consenso.
- A segurança em redes descentralizadas não vem de uma autoridade, mas do **consenso** da maioria dos nós e de **incentivos econômicos** alinhados.
- A evolução de simples registros de saldos (Bitcoin) para **Smart Contracts** (Ethereum), permitindo que códigos autoexecutáveis rodem de forma descentralizada.

? 6. Questões de Autoavaliação (Estilo Flashcard)

- Qual é o principal problema da computação financeira distribuída que a tecnologia Blockchain conseguiu resolver?

Ver Resposta O problema do Gasto Duplo (Double Spending). A blockchain evita que um

mesmo ativo digital seja gasto ou copiado mais de uma vez através do uso de um registro público imutável e regras de consenso entre os nós validadores.

- **Qual é a principal diferença arquitetural e filosófica entre a Web 2.0 e a Web 3.0?**

Ver Resposta A Web 2.0 possui uma arquitetura centralizada (cliente-servidor), onde o poder, as regras e os dados dos usuários pertencem a empresas corporativas. A Web 3.0 é descentralizada (P2P/Blockchain), focada na posse de dados pelo usuário, operando sem a necessidade de um servidor ou entidade central confiável.

- **Por que é praticamente impossível alterar uma transação antiga em uma Blockchain?**

Ver Resposta Porque cada bloco armazena o Hash do bloco anterior. Qualquer alteração em um dado antigo mudaria o Hash desse bloco, invalidando automaticamente todos os blocos gerados depois dele. A rede rapidamente detectaria a anomalia através do consenso e a rejeitaria.

- **Como os validadores de uma rede Blockchain são desencorajados a cometer fraudes?**

Ver Resposta Através de incentivos econômicos (criptomoedas/tokens). Para validar transações, eles precisam gastar recursos reais (como energia elétrica computacional ou deixando moedas em garantia). Se tentarem fraudar, os outros nós rejeitarão o bloco fraudulento e o validador malicioso perderá seu investimento e não receberá a recompensa.

Aula 2

Disciplina: Blockchain e Web 3.0 **Vídeo de Referência:** [Aula 2 - Introdução à Blockchain \(Parte 2\).mp4](#)

1. Visão Geral e Objetivos

- **Contexto:** Nesta aula, o professor apresenta uma visão detalhada da estrutura geral da disciplina e aprofunda conceitos vitais sobre o funcionamento das redes Blockchain, especialmente sobre arquitetura de redes e mecanismos de consenso. O entendimento desses pilares é fundamental para entender como redes descentralizadas mantêm segurança sem uma autoridade central.
- **Objetivos de Aprendizagem:** O aluno deve ser capaz de diferenciar modelos de consenso (PoW vs PoS), entender a estrutura de disciplinas que envolvem a Web3 e compreender o que são Smart Contracts e como o processamento computacional ocorre e é cobrado nessas redes.

2. Conceitos-Chave e Definições

- **Proof of Work (PoW):** Mecanismo de consenso utilizado pelo Bitcoin que exige alto poder computacional (esforço físico e gasto elétrico) para resolver desafios matemáticos e ter o direito de validar um bloco.
- **Proof of Stake (PoS):** Mecanismo de consenso alternativo (adotado pelo Ethereum em sua atualização) onde a validação não depende de poder computacional, mas da quantidade de moedas (stake) que um participante possui bloqueadas na rede.
- **Smart Contracts:** Contratos inteligentes que são essencialmente códigos de programação autoexecutáveis armazenados na blockchain. Eles possuem a capacidade de manter estado, tomar decisões lógicas e transferir valor.
- **Ethereum Virtual Machine (EVM):** Máquina virtual responsável por executar os Smart Contracts na rede Ethereum de maneira descentralizada.
- **Consenso:** Processo pelo qual diversos nós (computadores) em uma rede peer-to-peer (P2P) concordam sobre qual é a versão verdadeira e válida do registro de transações.

3. Explicação Detalhada por Tópicos

- **Mecanismos de Consenso:**
 - O professor explicou que o Bitcoin se popularizou porque encontrou uma forma de redes P2P entrarem em consenso de forma confiável usando o **PoW**.
 - No PoW, quem tem mais poder de processamento (clusters de mineração) tem mais chances de minerar o bloco e ganhar a recompensa. Isso gerou um enorme problema ecológico (alto consumo de energia global).

- Devido a isso, redes mais modernas migraram para o **PoS**. No PoS, a "força" do validador não vem de supercomputadores gastando energia, mas sim do seu balanço financeiro travado na rede (os validadores "apostam" suas moedas para validar blocos).

- **Smart Contracts e Custo Computacional:**

- Diferente do Bitcoin, que apenas registra transferências financeiras, plataformas como o Ethereum rodam código de verdade.
- O código é escrito numa linguagem chamada **Solidity** e executado na **EVM**.
- Como rodar código exige processamento real de milhares de computadores, toda operação computacional custa uma taxa (frequentemente chamada de "Gas"). Se um contrato entra em loop infinito, ele consome a taxa do usuário até o dinheiro acabar e a transação ser abortada, protegendo a rede.

- **A imutabilidade dos Contratos:**

- Uma vez que um Smart Contract é implantado na blockchain, seu código não pode ser alterado. Qualquer falha lógica pode ser catastrófica, permitindo que cibercriminosos drenem todos os fundos mantidos pelo contrato.

4. Exemplos Práticos e Analogias

- **Analogia da Votação (Consenso):** Para explicar os diferentes tipos de consenso, o professor comparou o modelo a uma eleição. Em um modelo clássico, cada pessoa tem um voto. No modelo Proof of Stake, se você tiver R\$ 10, seu voto vale 10 vezes mais do que o de alguém que tem R\$ 1. Quem tem mais dinheiro investido na rede tem mais poder de validação, pois também tem mais a perder se a rede falhar.
- **Analogia do Sistema SWIFT vs Blockchain:** O sistema bancário tradicional internacional usa o SWIFT, fortemente controlado por órgãos americanos. Se eles desejarem, podem bloquear as transferências de um país. A Blockchain surge como uma alternativa similar a países tentando criar o "BRICS Pay": uma rede autônoma e imutável que não pode ser facilmente censurada por uma entidade central.

5. Resumo dos Pontos Críticos (O que vai cair na prova)

- A diferença entre **Proof of Work (PoW)**, que se baseia em esforço computacional e gasto elétrico, e **Proof of Stake (PoS)**, que se baseia em retenção de ativos (stake) e é mais eficiente.
- A definição e o papel fundamental do **Consenso**: garantir a veracidade dos dados em uma rede onde os participantes não confiam uns nos outros.
- O conceito de **Smart Contracts**, e o fato crítico de que eles são imutáveis após implantação, necessitando auditorias rigorosas.
- A função da **EVM** (Ethereum Virtual Machine) em processar as instruções lógicas do Solidity.

? 6. Questões de Autoavaliação (Estilo Flashcard)

- Qual é a principal crítica feita ao modelo Proof of Work (PoW) e como o Proof of Stake (PoS) tenta resolver isso?

Ver Resposta O PoW é criticado por seu altíssimo consumo de energia elétrica, já que exige hardware especializado fazendo cálculos constantes. O PoS resolve isso ao eliminar a necessidade de supercomputadores, baseando o direito de validar blocos na quantidade de tokens que o validador bloqueia na rede.

- O que é um Smart Contract e qual é a sua principal característica de segurança/risco após a publicação?

Ver Resposta É um código de programação autoexecutável hospedado na blockchain. Seu maior risco é a **imutabilidade**: uma vez publicado, não pode ser editado. Se houver um bug, hackers podem explorá-lo sem que o criador possa simplesmente "atualizar" o código original para interromper o ataque.

- O que acontece se um Smart Contract no Ethereum entrar em um "loop infinito" durante a execução?

Ver Resposta A rede não irá travar. O contrato consumirá a "taxa" (Gas) fornecida pelo usuário a cada operação. Quando a taxa acabar, a transação será interrompida (revertida) por falta de fundos, preservando a disponibilidade da EVM.

- O que significa dizer que uma rede não possui um nó central de confiança?

Ver Resposta Significa que nenhuma única entidade (como um banco ou servidor central) dita qual é a versão verdadeira do banco de dados. Em vez disso, a veracidade é garantida através do algoritmo de consenso operado de forma distribuída pelos nós da rede.

Aula 3

Disciplina: Blockchain e Web 3.0 **Vídeo de Referência:** @[aulas/videos/Aula 3 - Introdução à Blockchain (Parte 3).mp4]

🔗 1. Visão Geral e Objetivos

- **Contexto:** Esta aula aprofunda a introdução aos conceitos fundamentais da tecnologia Blockchain, detalhando suas características principais, como dados são persistidos e a anatomia de um bloco. O tema é essencial para compreender como a rede garante segurança, transparência e descentralização sem depender de uma entidade ou servidor central.
- **Objetivos de Aprendizagem:** O aluno deve ser capaz de identificar as propriedades fundamentais de uma blockchain (como imutabilidade e distribuição), compreender a estrutura interna de um bloco (Header, Hash, Timestamp, Nonce, Merkle Root) e entender conceitos práticos como exploradores de blocos e vetores de ataques comuns (como o ataque de reprodução).

🔑 2. Conceitos-Chave e Definições

- **Distributed Ledger (Livro-razão distribuído):** Banco de dados mantido e atualizado de forma descentralizada por múltiplos nós da rede, sem um administrador ou servidor centralizador.
- **Imutabilidade:** Propriedade que garante que, uma vez que uma informação é gravada e consolidada na blockchain, ela não pode ser alterada, modificada ou apagada.
- **Endereço (Carteira / Chave Pública):** Identificador público único do usuário na rede (comparado a um CPF ou número de conta bancária), utilizado para receber transações.
- **Chave Privada:** Senha criptográfica secreta que permite ao usuário autorizar transações e manipular os dados e fundos vinculados ao seu endereço na blockchain.
- **Block Explorer (Explorador de Blocos):** Ferramenta (como o Etherscan) que funciona como um motor de busca para a rede, permitindo visualizar transações, blocos minerados e endereços de uma blockchain pública em tempo real.
- **Timestamp:** Marcação de tempo computacional em formato numérico inteiro (geralmente segundos ou milissegundos desde a Era Unix) que registra e imortaliza o momento exato em que um bloco ou transação foi criado.
- **Parent Hash (Hash do Bloco Anterior):** Assinatura criptográfica do bloco imediatamente anterior, responsável por encadear os blocos de forma sequencial e formar a "corrente" (chain).
- **Nonce (Number used once):** Valor numérico aleatório ou sequencial utilizado uma única vez na validação e construção do bloco (mineração).
- **Merkle Root:** Raiz de uma árvore de hashes que sumariza, empacota e garante a integridade e autenticidade de todas as transações incluídas em um bloco.
- **Ataque de Reprodução (Replay Attack):** Ataque onde uma comunicação ou transação válida é interceptada e reenviada maliciosamente por terceiros.



3. Explicação Detalhada por Tópicos

- **Características Principais da Blockchain:**

- **Registro Distribuído:** Elimina-se o servidor central. Todos os nós participantes que mantêm a rede possuem uma cópia integral do livro-razão e aplicam regras de consenso para atualizá-la.
- **Dados Permanentes e Imutáveis:** A impossibilidade de retroagir e alterar dados torna a tecnologia um ambiente de extrema confiança (trustless), ideal para cenários sensíveis como registros médicos e votações eletrônicas.
- **Transparência e Rastreabilidade:** Em blockchains públicas, qualquer indivíduo pode inspecionar e rastrear qualquer transação de ponta a ponta. A privacidade é mantida através do pseudo-anonimato: você enxerga a string do endereço, mas não sabe necessariamente qual humano ou empresa o controla.
- **Segurança Criptográfica Avançada:** O uso estrito de chaves assimétricas garante que apenas o legítimo dono da chave privada consiga realizar débitos ou envios de seu respectivo endereço.

- **Novos Modelos de Negócio e Micropagamentos:**

- O modelo bancário convencional não suporta transferências de frações mínimas de centavos devido ao elevado custo por transação.
- Com o uso de contratos inteligentes e redes secundárias, a blockchain viabiliza **micropagamentos** de máquina para máquina. O professor dá o exemplo do código HTTP nativo `402 Payment Required`, que foi projetado para isso mas raramente usado por falta de infraestrutura na Web2, e que agora ganha força na Web3.

- **Estrutura e Anatomia de um Bloco:**

- A blockchain agrupa as requisições (transações) em blocos. O **Cabeçalho do Bloco (Block Header)** concentra as metainformações.
- O **Parent Hash** é a engrenagem que sela a imutabilidade: se alguém tentar falsificar um dado de um bloco antigo, o hash desse bloco mudará drasticamente (efeito avalanche). Consequentemente, o bloco seguinte (que guarda o Parent Hash) não reconhecerá a cadeia, quebrando a sequência e invalidando o ataque perante a rede.
- O **Timestamp** numérico no bloco elimina a necessidade de fusos horários humanos complexos, garantindo uma datação global padronizada e matemática daquela aprovação.

- **Ataques de Identidade e Reprodução (Replay Attack):**

- Na blockchain, a comunicação do seu computador para o nó validador viaja pela internet aberta.
- Se um usuário assina uma transação "Eu pago 10 reais para o João" e não insere nenhum elemento temporal/único, um hacker no meio do caminho captura esses bytes e os reenvia repetidas vezes, esgotando a carteira da vítima.

- O remédio para esse vetor de ataque é a adição de um **Timestamp** ou um **Nonce** serial no corpo da mensagem *antes* da assinatura, impossibilitando que a cópia idêntica da mensagem bruta seja acatada duas vezes pela rede.

4. Exemplos Práticos e Analogias

- **Analogia da Conta Bancária:** O *Endereço* é o seu número de conta e agência bancária — você imprime no seu cartão de visitas e passa para qualquer um. A *Chave Privada* é a sua senha de 6 dígitos e biometria juntas — você não compartilha com absolutamente ninguém e a utiliza unicamente para transferir valores.
- **Explorador Etherscan na Prática:** O professor abre ao vivo o *Etherscan* (explorador da Ethereum) para mostrar que a rede é como um auditório de vidro. Ele mostra blocos sendo produzidos em intervalos de 12 segundos e transferências, algumas valendo centavos e outras envolvendo quantias altíssimas, ocorrendo globalmente e visíveis a qualquer um com um navegador web.
- **O Problema do Replay Attack:** Imagine assinar um cheque em branco apenas com "Pague R\$ 100". O cobrador poderia tirar cópias do cheque e compensá-lo várias vezes. O número de série no talão de cheques funciona como o *Nonce* da transação blockchain, garantindo que aquela via específica seja compensada apenas uma vez.

5. Resumo dos Pontos Críticos (O que vai cair na prova)

- Uma Blockchain tem como base os pilares do **registro distribuído** (sem controle central), da **imutabilidade** (inviabilidade de exclusão ou edição) e do **pseudo-anonimato** (contas públicas rastreáveis, identidades ocultas).
- A relação de continuidade e proteção do passado baseia-se primordialmente no **Parent Hash** (Hash do Bloco Anterior).
- O controle de tempo é gerido por um **Timestamp** numérico de base computacional (ex. Inteiro Unix), não dependendo de convenções de formatação humanas como dd/mm/aaaa.
- A validação de quem está agindo na rede provém da **Chave Privada**, enquanto um **Ataque de Reprodução (Replay Attack)** é prevenido através de mecanismos de autenticação com dados descartáveis ou mutáveis no tempo (como Nonce e Timestamp).

? 6. Questões de Autoavaliação (Estilo Flashcard)

- **Pergunta 1:** Qual é a função da Chave Pública (Endereço) e da Chave Privada em uma rede Blockchain?

Ver Resposta A Chave Pública atua como um endereço ou identificador para recebimentos, visível publicamente. A Chave Privada atua como a senha que confere posse sob a conta, permitindo que o usuário assine e autorize o gasto dos saldos ou a interação com contratos inteligentes.

- **Pergunta 2:** O que torna a blockchain imutável ao longo do tempo? Qual componente do bloco é fundamental para isso?

Ver Resposta O componente fundamental é o "Hash do Bloco Anterior" (Parent Hash). Ele cria um elo criptográfico sequencial; se os dados de qualquer bloco antigo forem alterados, seu hash será completamente modificado, quebrando o elo com o bloco seguinte e invalidando a fraude perante o restante da rede.

- **Pergunta 3:** Como funciona a dicotomia entre transparência total e privacidade em blockchains públicas?

Ver Resposta A rede é 100% transparente pois permite que qualquer pessoa rastreie o saldo e as transações de todos os endereços via Exploradores de Blocos. Contudo, há privacidade na forma de pseudo-anonimato, pois a criação de um endereço não exige vínculo formal e obrigatório com os dados reais de um indivíduo (como nome ou CPF).

- **Pergunta 4:** O que é um Replay Attack (Ataque de Reprodução) e qual a técnica usada para mitigá-lo nas assinaturas de transações?

Ver Resposta O ataque ocorre quando um indivíduo intercepta a comunicação de uma transação perfeitamente válida e tenta reenviá-la à rede para executá-la repetidas vezes em proveito próprio. A mitigação é feita adicionando um carimbo de tempo (Timestamp) ou um número sequencial único (Nonce) no momento da assinatura, tornando qualquer cópia duplicada inválida e recusada pela rede.

Aula 4

Disciplina: Blockchain e Web 3.0 **Vídeo de Referência:** @[aulas/videos/Aula 4 - Introdução à Blockchain (Parte 4).mp4]

1. Visão Geral e Objetivos

- **Contexto:** Esta aula aborda o fluxo operacional e de segurança de uma transação em sistemas distribuídos, contrastando assinaturas digitais em ambientes centralizados (como Gov.br) com a criptografia assimétrica de chaves pública e privada em redes Blockchain. O tema é fundamental para compreender a soberania dos dados dos usuários e a gestão descentralizada de ativos digitais.
- **Objetivos de Aprendizagem:** O aluno deve compreender a mecânica matemática das assinaturas digitais, diferenciar a segurança de custódia própria versus centralizada (Exchanges) e entender os riscos e consequências da imutabilidade das transações em blockchain.

2. Conceitos-Chave e Definições

- **Assinatura Digital:** Mecanismo criptográfico que atesta a **autoria** e **integridade** de uma transação ou documento, garantindo que o emissor é realmente quem afirma ser e que os dados não foram modificados.
- **Criptografia Simétrica:** Método de cifragem onde a **mesma chave** é compartilhada entre emissor e receptor para criptografar e descriptografar dados.
- **Criptografia Assimétrica:** Sistema que utiliza um **par de chaves** matematicamente ligadas: uma **chave pública** (compartilhada com todos) e uma **chave privada** (mantida em segredo absoluto).
- **Endereço da Carteira:** A **chave pública** (ou uma representação formatada dela) que serve como identificador de destino para receber ativos na rede.
- **Chave Privada:** Chave criptográfica secreta que permite ao usuário **assinar transações** e autorizar a saída de ativos da sua respectiva chave pública.
- **Exchange (Corretora):** Plataforma centralizada que atua como **custodiante** das chaves privadas dos usuários, gerenciando as negociações em seu nome.
- **Certificado Digital:** Documento eletrônico emitido por uma **Autoridade Certificadora (AC)** de confiança que associa uma chave pública a uma identidade (ex: Gov.br).

3. Explicação Detalhada por Tópicos

- **Fluxo de Assinatura e Validação no Modelo Centralizado (Gov.br):**
 - O usuário envia um documento à plataforma centralizada para assinar.

- O sistema gera o **hash do conteúdo** e o acopla ao **certificado digital** do usuário, gerando uma assinatura digital auto-contida.
 - **Impossibilidade de clonagem:** Não é possível copiar a assinatura de um documento e colar em outro, pois o hash final é diretamente dependente do conteúdo exato do documento original. Qualquer alteração no texto gera um hash incompatível.
 - **A validação:** Para atestar a autenticidade, o documento assinado é submetido à **autoridade certificadora central** (como o governo), que recalcula o hash usando o certificado e valida se há correspondência.
- **Criptografia Assimétrica e Descentralização em Blockchain:**
 - Em redes Blockchain (P2P), **não existe um órgão central** ou governo para validar identidades ou certificar assinaturas.
 - A validação ocorre diretamente na rede por meio de algoritmos baseados em **matemática discreta e criptografia de curvas elípticas**.
 - A **chave pública** funciona de forma análoga a uma caixa de correio: é pública e serve apenas para receber fundos (qualquer pessoa pode inserir uma mensagem/moeda).
 - A **chave privada** é o único meio capaz de abrir a "caixa de correio" e retirar ou mover os ativos (gerar transações de saída).
 - Como a responsabilidade de manter a chave privada segura é **100% do usuário**, a perda dessa chave significa a perda irreversível do acesso aos fundos.
- **Custódia de Chaves e o Papel das Exchanges:**
 - O lema básico do ecossistema cripto é *"Not your keys, not your coins"* (se as chaves privadas não são suas, as moedas não são suas).
 - Ao utilizar **Exchanges** (como Nubank ou corretoras tradicionais), o usuário não possui controle direto da chave privada. A corretora atua como um **ente centralizador**, retendo as chaves privadas dos clientes e processando as transações na rede por eles.
 - **Vazamento e Fraude:** Se os servidores da Exchange forem hackeados, ou se a empresa agir de má-fé, as chaves privadas de todos os usuários podem ser comprometidas.
 - Devido à **imutabilidade e irreversibilidade** da blockchain, uma vez que uma transação fraudulenta é assinada com a chave privada e registrada no bloco, os fundos não podem ser recuperados ou estornados por nenhuma autoridade.

4. Exemplos Práticos e Analogias

- **Analogia da Caixa de Correio:** A **chave pública** é a fenda externa da caixa de correio na parede: qualquer pessoa que passa na rua pode colocar cartas por ali. A **chave privada** é a chave física que abre a portinha traseira da caixa pelo lado de dentro da casa, permitindo apenas ao proprietário retirar e ler as correspondências.
- **Analogia do Cartório Físico vs. Digital:** No modelo manual, a validação de assinatura exige ir fisicamente ao cartório comparar grafias com uma ficha de firma (suscetível a falhas humanas ou suborno). No digital, o cartório é substituído pela **Autoridade Certificadora** e pela **criptografia matemática**, tornando a checagem instantânea e inviolável.

- **Estudo de Caso do Jogador Gustavo Escarpa:** O jogador investiu milhões em uma corretora centralizada que prometia comprar criptomoedas. A empresa usou o dinheiro de forma fraudulenta e depois declarou falência alegando ter sofrido uma invasão hacker com roubo das chaves privadas. Como as transações em blockchain são irreversíveis, uma vez que as chaves custodiadas foram expostas e os saldos movidos, o dinheiro tornou-se irrecuperável.
- **O Caso do HD no Aterro Sanitário:** Um programador norueguês perdeu o acesso a centenas de milhões de dólares em Bitcoin porque descartou acidentalmente o disco rígido contendo sua **chave privada**. Devido à segurança criptográfica de 256 bits, é matematicamente impossível realizar engenharia reversa para descobrir a chave sem o backup das **12 palavras (seed phrase)**.

🔗 5. Resumo dos Pontos Críticos (O que vai cair na prova)

- A diferença entre o fluxo de assinatura digital **centralizado** (dependente de uma Autoridade Certificadora central) e o fluxo **descentralizado** (independente de servidor, baseado em criptografia assimétrica).
- A função matemática e operacional da **chave pública** (endereço da carteira) e da **chave privada** (autorização e assinatura de transações).
- O risco sistêmico de delegar a custódia das chaves privadas a terceiros (**Exchanges**).
- A **irreversibilidade** e **imutabilidade** da blockchain: transações assinadas por chaves privadas vazadas ou roubadas não podem ser desfeitas.
- A importância do backup próprio da chave privada (e a estrutura das **seed phrases** / 12 palavras) para a segurança e soberania dos ativos.

? 6. Questões de Autoavaliação (Estilo Flashcard)

- **Pergunta 1: Por que é impossível clonar uma assinatura digital de um documento PDF para outro?**

Ver Resposta Porque a assinatura digital é gerada a partir do hash matemático do conteúdo exato do documento original combinado ao certificado digital do emissor. Se a assinatura for copiada para outro arquivo, o hash gerado a partir do novo conteúdo não corresponderá à assinatura copiada, invalidando-a imediatamente.

- **Pergunta 2: Qual é a função da chave privada e o que acontece se o usuário perdê-la em uma rede descentralizada?**

Ver Resposta A chave privada serve para assinar digitalmente as transações, provando a propriedade dos ativos e autorizando a sua transferência. Caso o usuário a perca, ele perde permanentemente o acesso aos seus ativos. Como não há uma entidade centralizadora (suporte técnico ou banco) para redefinir credenciais, os fundos ficam congelados para sempre.

- **Pergunta 3: O que significa o ditado "Not your keys, not your coins" no contexto de criptoativos?**

Ver Resposta Significa que quem detém a chave privada de um endereço controla verdadeiramente os fundos contidos nele. Se você deixa suas criptomoedas sob custódia de uma Exchange (corretora), a chave privada fica com ela. Portanto, você possui apenas uma promessa de saldo em uma plataforma centralizada, e não a posse real dos ativos na blockchain.

- **Pergunta 4: Como um hacker que invade uma corretora centralizada (Exchange) consegue roubar as criptomoedas dos usuários de forma irreversível?**

Ver Resposta Ao invadir a infraestrutura da corretora, o hacker rouba as chaves privadas mantidas sob custódia dela. Com essas chaves, ele assina transações legítimas perante a blockchain enviando os fundos para seus próprios endereços. Como a blockchain é descentralizada e imutável, a rede aceita as assinaturas como válidas e a transação não pode ser desfeita por nenhuma autoridade.

- **Pergunta 5: Como a blockchain valida a autoria de uma transação enviada por um usuário sem um servidor central?**

Ver Resposta A validação ocorre por meio da criptografia assimétrica. O remetente assina a transação com sua chave privada. Os nós validadores da rede P2P usam a chave pública do remetente (endereço da carteira) para decifrar a assinatura e verificar matematicamente que ela foi gerada pela chave privada correspondente, garantindo a autoria sem expor a chave privada.

Aula 5

Disciplina: Blockchain e Web 3.0 **Vídeo de Referência:** @[aulas/videos/Aula 5 - Introdução à Blockchain (Parte 5).mp4]

🔑 1. Visão Geral e Objetivos

- **Contexto:** Esta aula aprofunda o papel das funções hash criptográficas na segurança da informação, examina os tipos de blockchain com base no controle de acesso e discute a escalabilidade de vazão de transações (TPS). Compreender esses conceitos é essencial para a tomada de decisão no design de arquiteturas centralizadas vs. descentralizadas.
- **Objetivos de Aprendizagem:** O aluno deve compreender as propriedades formais das funções hash (e suas aplicações práticas como checksum e armazenamento de senhas), classificar as diferentes topologias de blockchain (pública, privada, consórcio e híbrida) e entender os fatores de escalabilidade e throughput de redes tradicionais vs. tecnologias baseadas em grafos (Hedera Hashgraph).

🔑 2. Conceitos-Chave e Definições

- **Função Hash Criptográfica:** Função matemática que mapeia dados de qualquer tamanho para uma sequência de caracteres de tamanho fixo (digest). É determinística e de via única.
- **Efeito Avalanche:** Propriedade que garante que qualquer pequena alteração na entrada mude o hash resultante de forma imprevisível e drástica.
- **Resistência à Pré-Imagem:** Impossibilidade computacional de reverter um hash, ou seja, descobrir o dado original a partir da sua representação em hash.
- **Resistência à Colisão:** Propriedade que torna computacionalmente inviável encontrar duas entradas diferentes que produzam exatamente o mesmo hash.
- **SHA-256:** Função hash de 256 bits adotada como pilar de integridade na blockchain do Bitcoin.
- **Keccak-256:** Variante do SHA-3 de 256 bits usada na rede Ethereum para transações e contratos inteligentes.
- **RIPEMD-160:** Algoritmo que gera hashes de 160 bits, usado no Bitcoin para compactar endereços públicos.
- **Blockchain Pública:** Rede sem permissão (*permissionless*), transparente e auditável por qualquer participante.
- **Blockchain Privada:** Rede permissionada controlada por uma única organização que gerencia autorizações de leitura e validação.
- **Blockchain de Consórcio:** Rede semi-descentralizada controlada por um grupo predefinido de organizações parceiras com governança compartilhada.
- **Blockchain Híbrida:** Modelo que integra facilidades privadas (controle de acesso) e públicas (validação e auditoria externa).
- **Throughput (TPS):** Vazão de transações por segundo processadas e consolidadas por uma

rede distribuída.

- **Directed Acyclic Graph (DAG):** Estrutura de dados não linear (grafos) usada por redes como a Hedera Hashgraph para processamento paralelo, superando os gargalos de escalabilidade das blockchains lineares tradicionais.
- **HBAR:** Token nativo da rede Hedera Hashgraph usado para pagamento de taxas de transação e governança.

3. Explicação Detalhada por Tópicos

• Propriedades Críticas das Funções Hash:

- **Determinismo:** A mesma entrada precisa gerar exatamente a mesma saída a cada execução.
- **Unidirecionalidade (Via Única):** O cálculo deve ser fácil em uma direção, mas inviável na inversa. A segurança é matemática e não institucional.
- **Efeito Avalanche:** Pequenas perturbações na entrada mudam a representação do hash.
- **Resistência a Colisões:** Encontrar duas entradas com o mesmo hash compromete a segurança. Algoritmos antigos como MD5 e SHA-1 possuem falhas conhecidas de colisão, enquanto SHA-256 e Keccak-256 permanecem seguros.
- **Saída de Tamanho Fixo:** Independentemente de a entrada ser 1 caractere ou um arquivo de 1 GB, a saída mantém o mesmo tamanho (ex. 256 bits no SHA-256), facilitando comparações computacionais rápidas.

• Aplicações Práticas das Funções Hash:

- **Armazenamento de Senhas:** Aplicações seguras guardam apenas o hash das senhas dos usuários. No login, o hash da senha digitada é gerado e comparado com o armazenado no banco de dados. *(Nota: O conceito de **Salt**, que adiciona dados aleatórios antes de aplicar o hash para evitar ataques de dicionário, é mencionado e será detalhado na próxima aula).*
- **Checksum (Soma de Verificação):** Garante a integridade física de arquivos na internet. Downloads de sistemas operacionais fornecem assinaturas que o usuário pode comparar localmente para atestar que o arquivo não está corrompido ou malicioso.

• Taxonomia das Redes Blockchain:

- **Públicas:** Totalmente descentralizadas. Nós validadores operam em código aberto (*open source*). O repositório Go-Ethereum (*Geth*) no GitHub demonstra essa transparência. Exemplos: Bitcoin e Ethereum.
- **Privadas:** Uma única empresa gerencia os nós validadores. Tem foco em controle, privacidade interna e velocidade. Exemplo: Hyperledger Fabric.
- **Consórcio:** Um grupo de empresas se junta para governar a rede. Útil para cooperação sem confiança mútua. O professor cita o exemplo das empresas brasileiras de petróleo (Petrobras, Shell, Chevron, Total) cooperando em conjunto com a ANP para auditar o mercado de gás natural. Outros exemplos: R3 Corda, Energy Web Chain e BRICS Pay.

- **Híbridas:** Transações pequenas ocorrem em ambientes públicos de auditoria simples, e movimentações corporativas de alta sensibilidade rodam em rede restrita privada. Exemplo: Dragonchain.

- **TPS, Escalabilidade e a Tecnologia Hashgraph (Hedera):**

- **O Gargalo da Serialização:** Redes tradicionais processam transações sequencialmente (serialização), acumulando-as em blocos lineares, o que limita o TPS (Bitcoin processa ~3-7 TPS; Ethereum original ~12-15 TPS).
- **A Proposta da Hedera Hashgraph:** Utiliza a tecnologia DAG (grafo acíclico) e consenso de "fofoca sobre fofoca" (*gossip about gossip*) para processar transações de forma contínua e paralela.
- **Vantagens de Escalabilidade:** Hedera atinge mais de 10.000 TPS, com finalização ultrarrápida (3-5 segundos) e taxas de transação extremamente baixas (na casa de \$0.0001).
- **Governança:** A Hedera é administrada por um conselho de grandes multinacionais de tecnologia (Google, IBM, LG, Ubisoft). A moeda da rede é a HBAR.

4. Exemplos Práticos e Analogias

- **O Checksum do Debian e a Autenticidade da Fonte:** Ao baixar uma imagem ISO do Debian (~754 MB), o site fornece sua assinatura SHA-256 ou SHA-512. O usuário pode rodar a verificação no computador local. Se o hash local coincidir com o do site, o arquivo está íntegro. O professor ressalta: se o hacker controlar a fonte (site falso), ele pode fraudar a assinatura e o arquivo. Daí a importância de verificar a validade dos certificados SSL/TLS da página web original.
- **A Analogia do Consórcio de Petróleo:** Se a Petrobras mantivesse sozinha a base de dados de gás natural de todo o setor, as concorrentes (Shell, Chevron) não confiariam. Se a ANP mantiver um servidor centralizado, há pontos únicos de falha e auditoria restrita. A blockchain de consórcio compartilha os nós de validação entre os concorrentes e o regulador, garantindo segurança coletiva no registro de dados confidenciais sem expor os dados ao público.
- **A Analogia do Gargalo de Trânsito (Fila vs. Cruzamento Livre):** A blockchain tradicional funciona como uma fila única de pedágio onde cada carro (transação) precisa passar de forma sequencial por uma única cancela. A Hedera Hashgraph funciona como um cruzamento inteligente sem cancelas, onde os carros se movem em fluxos paralelos e contínuos, trocando informações em tempo real para evitar acidentes (conflitos) e aumentar drasticamente a vazão.

5. Resumo dos Pontos Críticos (O que vai cair na prova)

- As **cinco propriedades fundamentais das funções hash** (determinismo, unidirecionalidade, efeito avalanche, resistência à colisão e tamanho fixo).
- A diferença entre **blockchain pública** (sem permissão, open source, auditável), **privada** (controle central) e **consórcio** (semi-descentralizada, governança corporativa compartilhada).
- O funcionamento e a importância do **Checksum** para integridade de arquivos e o papel dos

certificados digitais de segurança na autenticação de sua origem.

- O **Throughput (TPS)** como limitador de escalabilidade de blockchains sequenciais, contrastando com a tecnologia **Hashgraph (DAG)** que valida transações em paralelo.

? 6. Questões de Autoavaliação (Estilo Flashcard)

- **Pergunta 1: Como o Efeito Avalanche auxilia na integridade de dados salvos em uma Blockchain?**

Ver Resposta O Efeito Avalanche garante que qualquer mínima alteração de dados em um bloco anterior mude completamente o hash resultante do bloco. Como cada bloco subsequente aponta para o hash do bloco anterior, toda a cadeia a partir do ponto alterado é invalidada imediatamente. Isso impede que dados antigos sejam editados de forma fraudulenta sem que a rede perceba.

- **Pergunta 2: Qual é a distinção de governança entre a blockchain de consórcio e a blockchain privada?**

Ver Resposta Na blockchain privada, uma única organização controla totalmente a governança, as regras de negócios e a validação das transações. Na blockchain de consórcio, a validação e a governança são distribuídas de forma compartilhada entre múltiplas organizações pré-selecionadas (ex. empresas de um setor e agências reguladoras), evitando a centralização em um único participante.

- **Pergunta 3: O que é o Checksum e qual é a sua limitação se a fonte de validação estiver comprometida?**

Ver Resposta O Checksum é a soma de verificação por hash que valida se um arquivo não foi modificado. A limitação ocorre se a fonte que exibe o checksum (como o site de download) estiver comprometida ou sob ataque de falsificação (ex. DNS spoofing). O hacker pode alterar tanto o arquivo quanto o hash exibido na página, enganando o usuário. Por isso, a validação exige também a verificação de certificados SSL/TLS da página web.

- **Pergunta 4: Por que a Hedera Hashgraph consegue atingir mais de 10.000 TPS enquanto a blockchain do Bitcoin processa cerca de 3-7 transações por segundo?**

Ver Resposta O Bitcoin processa transações de forma sequencial e em blocos lineares e rígidos, gerando filas de espera de processamento (serialização). A Hedera Hashgraph usa uma estrutura em grafo (DAG) sem formação de blocos rígidos, o que permite processar e validar transações em paralelo de forma contínua usando o consenso distribuído de "fofoca sobre fofoca" (gossip about gossip).

Aula 6

Disciplina: Blockchain e Web 3.0 **Vídeo de Referência:** @[aulas/videos/Aula 6 - Introdução à Blockchain (Parte 6).mp4]

🔑 1. Visão Geral e Objetivos

- **Contexto:** Esta aula conclui a introdução conceitual da blockchain, estabelecendo os critérios objetivos para decidir entre o uso de redes distribuídas e bancos de dados tradicionais. São explorados casos práticos de aplicação, o funcionamento real dos NFTs como credenciais de acesso, a tokenização de ativos físicos e as principais desvantagens e gargalos da tecnologia.
- **Objetivos de Aprendizagem:** Ao final da aula, o aluno deve ser capaz de aplicar a regra prática de seleção de bancos de dados versus blockchain, explicar o mecanismo criptográfico de NFTs e de tokenização fracionada de ativos, e discutir criticamente os desafios de escalabilidade, sustentabilidade energética e conformidade legal (LGPD/GDPR) de sistemas distribuídos.

🔑 2. Conceitos-Chave e Definições

- **Dilema de Escolha (Regra Prática):** Critério triplo para decidir a adoção de blockchain: necessidade de **desconfiança mútua** entre as partes, obrigatoriedade de **histórico imutável** de dados e **ausência de autoridade central** aceitável.
- **CRUD (Create, Read, Update, Delete):** Operações básicas de manipulação de dados em bancos tradicionais. A blockchain suporta nativamente apenas operações de criação (*Create*) e leitura (*Read*).
- **Imutabilidade e Reversibilidade:** Características da rede em que registros consolidados nunca são apagados; erros são corrigidos apenas gerando-se **transações de compensação** (inversas).
- **Tokenização de Ativos Reais (RWA):** Processo de fragmentar a propriedade de um ativo físico (imóvel, passe de atleta, etc.) em frações digitais representadas por tokens em uma blockchain.
- **NFT (Non-Fungible Token):** Token exclusivo e indivisível em blockchain que serve como **credencial de propriedade** digital ou física, cuja autenticidade é atestada pela chave privada do detentor.
- **Escalabilidade (TPS):** A métrica de Transações por Segundo. Avalia a capacidade de vazão da rede frente a sistemas centralizados (ex: Visa).
- **Proof of Work (PoW) - Custo Energético:** Mecanismo de consenso que exige mineração computacional intensa, resultando em elevado consumo de eletricidade.
- **Interoperabilidade:** Capacidade de diferentes redes blockchains distintas (como Bitcoin e Ethereum) trocarem dados e ativos de forma direta e segura.



3. Explicação Detalhada por Tópicos

- **Quando Escolher Blockchain vs. Bancos de Dados Tradicionais:**

- **Regra Prática de Decisão:** Deve-se usar blockchain **apenas** quando os três critérios a seguir forem verdadeiros:
 1. **Múltiplas partes envolvidas** não confiam uma na outra.
 2. O **registro histórico de dados** precisa ser garantidamente imutável e protegido contra reescrita.
 3. **Não existe uma autoridade centralizadora** aceitável ou confiável para todos os envolvidos.
- Se esses critérios não forem atendidos, ou se a aplicação requerer a deleção física de registros (operações de exclusão), o uso de um **banco de dados tradicional (SQL/NoSQL)** é a solução mais econômica, performática e recomendada.

- **Casos Práticos de Aplicação e Tokenização de Ativos:**

- **Prontuários Médicos (SUS):** A blockchain garante a integridade forense de registros de atendimentos. Impede que hospitais ou médicos alterem dados retroativamente no sistema para encobrir erros profissionais em investigações de óbito.
- **NFTs como Credenciais:** A imagem digital associada a um NFT (como o macaquinho do *Bored Ape*) é pública e copiável. A verdadeira utilidade do NFT é funcionar como uma **credencial digital exclusiva** e infalsificável. O detentor prova a posse do token na portaria de clubes de luxo ou eventos assinando um desafio criptográfico com sua chave privada.
- **Tokenização e Fragmentação Imobiliária:** Representação de um imóvel físico em frações digitais. Um imóvel de R\$ 1 milhão pode ser dividido em 1 milhão de tokens. Investidores do mundo todo podem comprar pequenas frações do ativo e receber repasses de aluguel diretamente em suas carteiras em tempo real, sem intermediários. Esse conceito também se aplica a passes de jogadores de futebol.

- **Desvantagens e Limitações Sistêmicas da Blockchain:**

- **Gargalo de Escalabilidade (TPS):** Blockchains descentralizadas tradicionais apresentam baixo throughput. O Bitcoin processa em média **7 TPS**, enquanto a rede de pagamentos centralizada da Visa atinge mais de **24.000 TPS**.
- **Consumo de Energia (PoW):** Redes baseadas em Proof of Work consomem um volume colossal de energia elétrica (estima-se que a mineração mundial consome cerca de **1% da eletricidade global**). Isso motivou a migração de projetos para Proof of Stake (PoS) ou Proof of Authority (PoA).
- **Conflito com Leis de Privacidade (LGPD/GDPR):** O caráter permanente e imutável da blockchain choca-se diretamente com o **direito ao esquecimento** e a obrigação de exclusão de dados pessoais exigidos pela LGPD e GDPR.
- **Barreiras de Educação e Interoperabilidade:** A complexidade da gestão de chaves

privadas dificulta a adoção por usuários comuns. Além disso, as blockchains operam de forma isolada, demandando pontes complexas e vulneráveis para viabilizar a comunicação entre diferentes redes.

4. Exemplos Práticos e Analogias

- **A Analogia do Consórcio do BRICS Pay:** Para gerenciar transações financeiras entre nações soberanas parceiras (BRICS), nenhum país aceitaria delegar o banco de dados centralizado para um servidor controlado por outro país. A blockchain resolve isso: cada país opera nós validadores da rede de consórcio, garantindo consenso compartilhado sem centralização do poder técnico.
- **A Analogia do Git Revert:** Para explicar a reversibilidade, o professor compara a blockchain ao sistema Git. Se um desenvolvedor comete um erro e envia para o repositório, ele não apaga o histórico de commits. Em vez disso, ele realiza um `git revert`, adicionando uma nova transação que desfaz a ação anterior, mantendo todo o histórico visível.
- **Analogia do Ingresso de Show com Captura de Tela:** Ingressos tradicionais baseados em imagens estáticas de QR Code são facilmente duplicados e vendidos por cambistas a múltiplas pessoas. Um ingresso emitido como NFT impossibilita a cópia de posse: apenas o real proprietário detém a chave privada capaz de assinar a transação que autentica o ingresso no leitor da portaria.

5. Resumo dos Pontos Críticos (O que vai cair na prova)

- Os **três critérios essenciais** para justificar a escolha de blockchain em detrimento de bancos de dados tradicionais.
- A inexistência de operações de exclusão física (*Delete*) na blockchain; correções ocorrem apenas por **novas transações de reversão**.
- O conflito regulatório entre a **imutabilidade perpétua da blockchain** e as leis de privacidade (LGPD/GDPR) no direito ao esquecimento.
- O funcionamento operacional de **NFTs como credenciais criptográficas** e a **tokenização fracionada de ativos reais (RWA)**.
- O gargalo energético do algoritmo de consenso **Proof of Work** comparado às alternativas como Proof of Stake.

? 6. Questões de Autoavaliação (Estilo Flashcard)

- **Pergunta 1: Sob quais três condições é justificável optar pelo uso de uma blockchain em vez de um banco de dados tradicional?**

Ver Resposta O uso de blockchain se justifica quando: 1) múltiplas partes que não confiam mutuamente precisam operar no mesmo sistema; 2) o histórico dos registros deve ser garantidamente imutável; e 3) não existe uma autoridade central aceitável por todos para gerenciar os dados.

- **Pergunta 2: Como a blockchain lida com a correção de registros enviados incorretamente, dado que é imutável?**

Ver Resposta A blockchain não permite alterar ou deletar registros passados. Qualquer correção exige a emissão de uma nova transação com o efeito contrário (transação de compensação) para balancear a anterior. O histórico mantém registrados tanto o lançamento incorreto original quanto a sua devida correção subsequente.

- **Pergunta 3: Qual o principal problema das blockchains baseadas em Proof of Work (PoW) que gerou a migração para outros consensos?**

Ver Resposta O altíssimo consumo de energia elétrica. Como os mineradores precisam competir resolvendo problemas matemáticos de força bruta com poder de processamento computacional ininterrupto, essas redes geram um impacto ambiental drástico (consumindo historicamente cerca de 1% da eletricidade global), forçando a adoção de consensos mais ecológicos como o Proof of Stake (PoS).

- **Pergunta 4: Se qualquer pessoa pode fazer o download e salvar a imagem digital associada a um NFT, por que o detentor do NFT ainda é considerado o único proprietário?**

Ver Resposta Porque a imagem digital em si é apenas a representação visual. A blockchain registra a propriedade do token criptográfico unicamente vinculado àquela imagem. Apenas o real proprietário possui a chave privada correspondente que permite transferir o token ou assinar desafios criptográficos, provando ser o detentor da credencial válida.

- **Pergunta 5: Como a imutabilidade da blockchain entra em conflito direto com as regulamentações de proteção de dados como a LGPD brasileira e a GDPR europeia?**

Ver Resposta Ambas as leis garantem aos cidadãos o "direito ao esquecimento", obrigando sistemas a excluírem dados pessoais quando solicitado pelo usuário. Como a blockchain é projetada estruturalmente para impossibilitar a remoção ou alteração física de dados uma vez registrados no livro-razão, torna-se tecnicamente inviável apagar dados pessoais que tenham sido gravados diretamente em seus blocos.

Aula 7

Disciplina: Blockchain e Web 3.0 **Vídeo de Referência:** @[aulas/videos/Aula 7 - Introdução à Web 3.0 (Parte 1).mp4]

🔑 1. Visão Geral e Objetivos

- **Contexto:** Esta aula marca a transição da teoria de redes distribuídas para o desenvolvimento prático na Web 3.0. O foco central é compreender a arquitetura que integra aplicações web com a blockchain por meio de carteiras descentralizadas, além de distinguir os conceitos de corretoras centralizadas, finanças descentralizadas e ambientes de desenvolvimento.
- **Objetivos de Aprendizagem:** O aluno deve compreender a arquitetura de uma aplicação Web 3.0, diferenciar custódia centralizada (CEX) de descentralizada (DEX), aplicar os conceitos de on-ramp e off-ramp, entender o papel das Testnets no ciclo de desenvolvimento e discernir a diferença técnica entre *coins* nativas e *tokens* gerados por contratos inteligentes.

🔑 2. Conceitos-Chave e Definições

- **Web 3.0 (Web3):** Nova geração da internet que integra aplicações web diretamente à blockchain via carteiras descentralizadas no navegador, eliminando a dependência de servidores centrais e custodiantes terceiros.
- **Wallet Descentralizada (ex: MetaMask):** Ferramenta (comumente extensão de navegador) que gerencia as chaves públicas e privadas do usuário localmente, permitindo interagir e assinar transações em aplicações Web 3.0.
- **CEX (Centralized Exchange):** Corretora centralizada (como a Binance) que gerencia a compra/venda de criptoativos e detém a custódia das chaves privadas dos usuários.
- **DEX (Decentralized Exchange):** Protocolo descentralizado (como a Uniswap) que permite a troca de ativos diretamente na blockchain via contratos inteligentes, sem custódia de intermediários.
- **On-Ramp:** Processo de conversão de dinheiro fiduciário (tradicional, como o Real via Pix) em criptoativos e sua inserção na blockchain.
- **Off-Ramp:** Processo inverso de retirar criptoativos da blockchain e convertê-los de volta em dinheiro tradicional em uma conta bancária.
- **Mainnet:** A rede blockchain principal de produção, onde as transações são reais e os ativos possuem valor econômico de mercado.
- **Testnet (ex: Sepolia):** Rede blockchain de testes idêntica à Mainnet, usada para desenvolvimento, que opera com moedas fictícias sem valor financeiro.
- **Coin (Moeda Nativa):** Criptoativo nativo de uma blockchain (como o Ether no Ethereum), usado obrigatoriamente para pagar as taxas de processamento da rede (gas).
- **Token:** Ativo digital programável criado sobre uma blockchain existente por meio de contratos inteligentes (como tokens de utilidade, governança ou NFTs).



3. Explicação Detalhada por Tópicos

- **Arquitetura da Web 3.0 e Conexão de Carteiras:**

- No modelo Web 2.0, o usuário se conecta via login/senha tradicionais e os dados ficam sob a custódia de servidores corporativos centralizados.
- Na Web 3.0, a interface do usuário (front-end no navegador) conecta-se diretamente à blockchain usando uma **wallet descentralizada (MetaMask)** instalada como extensão.
- A identidade e a autorização de transações ocorrem localmente no computador do usuário, onde a extensão solicita a assinatura digital da transação via chave privada.
- O professor exemplifica com o caso da **Uncade** (plataforma de distribuição de jogos descentralizada, concorrente da Steam). Ao contrário do Steam (onde o dinheiro e os jogos ficam sob controle e regras da empresa), na Uncade todas as transações e posses de itens dos jogos são registradas na blockchain e pertencem exclusivamente à carteira local do jogador.

- **Exchanges Centralizadas (CEX) vs. Descentralizadas (DEX):**

- **Funcionamento de CEX (ex: Binance):** Permite a conversão fácil de moeda fiduciária para cripto (On-Ramp) via Pix. No entanto, ela atua sob o modelo de **custódia de chaves**: o usuário não tem a chave privada de sua conta. Se a corretora quebrar ou for hackeada, os fundos desaparecem ("Not your keys, not your coins").
- **Funcionamento de DEX (ex: Uniswap):** É uma aplicação puramente Web 3.0. O usuário apenas conecta sua carteira MetaMask ao site e realiza swaps (troca de moedas, como ETH por Stablecoins de dólar) diretamente na blockchain por meio de contratos inteligentes de liquidez, mantendo total controle de sua chave privada.

- **Ambientes de Desenvolvimento: Mainnet vs. Testnet:**

- Executar contratos e transações diretamente na **Mainnet** custa taxas reais de rede (*gas*), inviabilizando testes e depurações de códigos em desenvolvimento.
- Por isso, utilizam-se as **Testnets** (como a rede **Sepolia** do Ethereum). Elas emulam todas as funcionalidades da rede real, mas operam com moedas fictícias obtidas gratuitamente em sites chamados *faucets*.
- **Diferenciação das redes:** Dinheiro de teste da Testnet nunca pode ser migrado ou usado na Mainnet. O professor esclarece que os **nós validadores das duas redes são completamente diferentes**, operando sob chaves e identificadores de rede (Chain IDs) distintos que isolam as transações de cada ambiente.

- **Diferença entre Moedas (Coins) e Tokens:**

- **Moeda Nativa (Coin):** É a moeda de curso oficial e intrínseca da blockchain (ex: ETH no Ethereum, BTC no Bitcoin). Ela é a única aceita pelos validadores para pagar as taxas de rede (*gas*).
- **Tokens:** São ativos adicionais programados e implantados por meio de contratos inteligentes (como representações de tokens de dólares, reais ou NFTs). Embora

transacionem na rede, suas transferências exigem o pagamento de taxas cobradas na moeda nativa da blockchain correspondente (ex: para transferir um token ERC-20 na Ethereum, paga-se a taxa em ETH).

4. Exemplos Práticos e Analogias

- **O Caso do Funcionário Desistente na Uncade:** O professor relata um caso real em que os recebíveis das vendas de um jogo iam para a carteira de quem o havia cadastrado na plataforma Uncade. O funcionário que realizou o cadastro desligou-se da empresa e sumiu por uma semana. Como a plataforma era Web 3.0 e as chaves privadas pertenciam ao ex-funcionário, nem os administradores da Uncade nem o dono da empresa conseguiram acessar os \$7.000 acumulados na carteira dele. Foi necessário localizá-lo para que ele fizesse o login com sua wallet e assinasse a transação de transferência da titularidade dos recebíveis.
- **Demonstração Prática na Uniswap:** O professor demonstra a interface do protocolo Uniswap conectado à sua MetaMask na rede de teste Sepolia. Ele mostra que a aplicação lê o saldo de sua carteira local (5 ETH de teste) e permite estimar o swap para uma moeda pareada em dólar (com cotação simulada no momento). A troca não exige cadastro, e-mail ou intermediário, apenas a assinatura da MetaMask.
- **Analogia do On-Ramp e Off-Ramp (A Rampa de Acesso):** Imagine a blockchain como uma rodovia elevada (plataforma digital). O **On-Ramp** é a rampa de acesso que permite que o motorista suba com seu carro tradicional (dinheiro físico) para a rodovia convertendo-o em cripto. O **Off-Ramp** é a rampa de saída que permite descer da rodovia elevada, convertendo a criptomoeda de volta em dinheiro tradicional na sua conta bancária.

5. Resumo dos Pontos Críticos (O que vai cair na prova)

- A arquitetura da **Web 3.0** (Integração: Front-end Web $\rightarrow$ Extensão de Wallet/MetaMask $\rightarrow$ Blockchain).
- O risco inerente à custódia de chaves em **corretoras centralizadas (CEX)** versus a soberania das chaves em **protocolos descentralizados (DEX)**.
- O fluxo operacional de conversão de ativos por meio de **On-Ramp** e **Off-Ramp**.
- O isolamento absoluto e a finalidade das redes **Mainnet** (produção com valor real) e **Testnets** (desenvolvimento com valores fictícios).
- A diferenciação técnica entre **Coin** (moeda de taxas nativa) e **Token** (ativo lógico de contrato inteligente).

? 6. Questões de Autoavaliação (Estilo Flashcard)

- **Pergunta 1: Como o login e a identificação do usuário em uma aplicação Web 3.0 diferem do modelo tradicional da Web 2.0?**

Ver Resposta Na Web 2.0, o login depende de credenciais armazenadas em servidores centralizados (e-mail/senha ou contas como Google/Facebook). Na Web 3.0, não há contas em

servidores; a identidade do usuário é representada pelo endereço público da sua wallet (como MetaMask) e as ações são validadas a partir de assinaturas criptográficas geradas localmente pela sua chave privada.

- **Pergunta 2: Por que uma exchange centralizada (CEX) não representa o conceito puro de descentralização e quais são os riscos associados?**

Ver Resposta Porque uma CEX funciona como um custodiante tradicional (um banco): ela retém a posse das chaves privadas dos ativos de seus clientes. O risco é que, se a corretora sofrer insolvência, ataque hacker ou fraude interna, os usuários perderão o acesso ao dinheiro, uma vez que não possuem as chaves privadas para resgatar os ativos diretamente na blockchain.

- **Pergunta 3: Qual é o propósito da Testnet (como a Sepolia) e como a rede impede que criptomoedas de teste sejam utilizadas de forma fraudulenta na Mainnet?**

Ver Resposta O propósito da Testnet é permitir que desenvolvedores testem seus contratos inteligentes e aplicações Web 3.0 sem gastar dinheiro real com as taxas de transação (*gas*). As moedas de teste são separadas da rede principal porque a Testnet opera com um conjunto de nós validadores completamente independente, além de ter identificadores de rede (Chain IDs) e chaves de consenso distintos da Mainnet, tornando as moedas de teste inválidas e invisíveis na rede principal.

- **Pergunta 4: O que são On-Ramp e Off-Ramp no mercado de criptoativos e por que eles são necessários?**

Ver Resposta On-Ramp é a conversão de moedas fiduciárias (como o Real) em criptoativos para inserção na rede blockchain. Off-Ramp é o inverso: retirar criptoativos da blockchain e convertê-los de volta em dinheiro tradicional. Ambos são necessários porque servem de ponte entre a economia tradicional centralizada e o ecossistema financeiro descentralizado da blockchain.

- **Pergunta 5: Se um programador cria um novo ativo digital via contrato inteligente na rede Ethereum, esse ativo é considerado uma Coin ou um Token? Qual a moeda usada para pagar suas transações?**

Ver Resposta É considerado um Token, pois foi implementado por meio de lógica de contrato inteligente e não é a moeda base nativa da rede. Toda transação ou movimentação desse token deverá pagar as taxas de rede (gas) utilizando a Coin oficial da rede, que no caso da blockchain Ethereum é o Ether (ETH).

Aula 8

Disciplina: Blockchain e Web 3.0 **Vídeo de Referência:** @[aulas/videos/Aula 8 - Introdução à Web 3.0 (Parte 2).mp4]

🔑 1. Visão Geral e Objetivos

- **Contexto:** Esta aula é focada na preparação completa e configuração detalhada do ambiente local de desenvolvimento para a Web 3.0. O objetivo é instalar as dependências necessárias, configurar a depuração integrada entre a IDE e o navegador Chrome, criar a carteira digital na MetaMask e rodar um nó simulador de blockchain local através do Hardhat.
- **Objetivos de Aprendizagem:** O aluno deve ser capaz de instalar e gerenciar versões do Node.js, configurar o VS Code e o Google Chrome para depuração remota via console integrado, configurar e gerenciar chaves públicas/privadas na carteira MetaMask, simular uma rede blockchain com o Hardhat em sua máquina local e importar contas ricas para transações de testes.

🔑 2. Conceitos-Chave e Definições

- **NVM (Node Version Manager):** Ferramenta de linha de comando que gerencia múltiplas versões instaladas do Node.js, permitindo alternar de versão conforme a necessidade de cada projeto.
- **Solidity (by Nomic Foundation):** Extensão oficial do VS Code para desenvolvimento em Solidity que oferece suporte à linguagem e ao ecossistema Hardhat (realce de sintaxe, formatação e verificação de erros).
- **Launch Configuration (launch.json):** Arquivo de configuração do VS Code localizado na pasta `.vscode/` que estabelece as regras e portas de rede para depuração de aplicativos.
- **Remote Debugging Port:** Configuração que expõe uma porta de comunicação em uma instância do Chrome (ex: porta `9222`) para permitir que o depurador do VS Code interaja diretamente com o console do navegador.
- **Alias (Apelido):** Comando curto e personalizado configurado no terminal do sistema operacional que simplifica a execução de linhas de comando extensas e complexas.
- **Secret Recovery Phrase (Seed Phrase):** Frase de recuperação secreta composta por 12 palavras aleatórias usada como semente criptográfica para gerar a chave privada e recuperar a carteira digital.
- **Import Account (Importar Conta):** Ação de adicionar à carteira local (MetaMask) o controle de uma conta externa já existente na blockchain a partir de sua **chave privada** de texto hexadecimal.
- **Hardhat:** Framework de desenvolvimento em JavaScript/TypeScript para a rede Ethereum que provê ferramentas para compilação, deploy e simulação local de blockchain.
- **Nó Local de Blockchain (Hardhat Node):** Blockchain emulada e executada na máquina de

desenvolvimento (porta 8545) que fornece carteiras fictícias e simula o processamento e a validação de transações sem custos reais.

- **Chain ID (ID de Rede):** Número de identificação único associado a cada rede blockchain (ex: 31337 para a blockchain de testes local do Hardhat) para garantir a segurança e evitar transações replicadas em ambientes incorretos.

3. Explicação Detalhada por Tópicos

- **Gerenciamento do Node.js e Instalações no Sistema:**

- O professor inicia a aula orientando sobre a importância do controle de versões no Node.js.
- O uso do **NVM (Node Version Manager)** é recomendado por dar flexibilidade ao desenvolvedor de alternar entre versões sem gerar conflitos no sistema operacional. Para Linux/WSL, a instalação do Node e npm também pode ser realizada diretamente via gerenciador de pacotes (`sudo apt install nodejs npm`).

- **Extensões Fundamentais para Desenvolvimento no VS Code:**

- O setup do editor exige extensões vitais para o fluxo de trabalho da disciplina:
 - **ETH over:** Extensão para interações e consultas na rede Ethereum.
 - **Prettier - Code formatter:** Automatiza a formatação do código em padrões consistentes.
 - **Mermaid Preview:** Renderiza diagramas de fluxos desenhados em Markdown.
 - **Solidity (da Nomic Foundation):** Essencial para realce de sintaxe, linting de contratos inteligentes e integração nativa com o Hardhat.

- **Depuração Integrada com Chrome Remote Debugging:**

- A fim de agilizar o desenvolvimento e evitar a mudança constante de janelas para inspecionar erros, configura-se a depuração remota do Google Chrome diretamente no VS Code.
- O Chrome deve ser iniciado via linha de comando expondo a porta de depuração:
`google-chrome --remote-debugging-port=9222`.
- Cria-se um arquivo de inicialização [launch.json](file:///Ubuntu/home/victorhugo/cc-blockchain-e-web-3-2026.1/aulas/pratica/.vscode/launch.json) dentro da pasta `.vscode/` com as seguintes propriedades:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "chrome",
      "request": "attach",
      "name": "Chrome: anexar (porta 9222)",
    }
  ]
}
```

```
    "port": 9222,  
    "webRoot": "${workspaceFolder:Codes}/aula-1/frontend",  
    "urlFilter": "http://localhost:3000/*"  
  }  
]  
}
```

- Executando esse perfil de debug no VS Code, todas as mensagens, saídas de erro e logs do console do Chrome são capturados e impressos diretamente no painel **Debug Console** da IDE.

- **Instalação da Carteira MetaMask e Guarda de Chaves:**

- Demonstra-se o download e a ativação da extensão **MetaMask** na Chrome Web Store.
- O professor executa o passo a passo de criação de uma nova carteira local (com senha de no mínimo 8 caracteres) e explica a segurança da frase secreta de recuperação de 12 palavras.
- Enfatiza-se que as 12 palavras representam o único modo de restabelecimento do saldo e das chaves caso o dispositivo físico seja danificado ou formatado.

- **Inicialização e Simulação da Blockchain Local com o Hardhat:**

- Para simular o ecossistema descentralizado, cria-se na raiz do projeto o arquivo [hardhat.config.js](file:///Ubuntu/home/victorhugo/cc-blockchain-e-web-3-2026.1/aulas/pratica/aula-1/hardhat.config.js):

```
require("@nomicfoundation/hardhat-toolbox");  
  
module.exports = {  
  solidity: "0.8.20",  
  defaultNetwork: "hardhat",  
  networks: {  
    hardhat: {},  
    localhost: {  
      url: "http://127.0.0.1:8545",  
      chainId: 31337,  
    },  
  },  
};
```

- O arquivo [package.json](file:///Ubuntu/home/victorhugo/cc-blockchain-e-web-3-2026.1/aulas/pratica/aula-1/package.json) gerencia os scripts de automação. Executando **npm run chain** (que por baixo roda **npx hardhat node**), inicializa-se a simulação local da blockchain.
- O nó local do Hardhat cria 20 carteiras de teste contendo **10.000 ETH** fictícios cada uma, mostrando no terminal tanto a chave pública quanto a chave privada de cada conta.

• Configuração de Rede Customizada e Importação de Conta:

- Copia-se a chave privada da primeira conta (`Account #0`) gerada no terminal do Hardhat e importa-se na MetaMask utilizando a opção "Importar Conta" (colando a string hexadecimal).
- Adiciona-se uma rede personalizada na MetaMask informando os parâmetros do nó Hardhat:

■ **Network Name:** Hardhat Local (ou nome à escolha)

■ **RPC URL:** `http://127.0.0.1:8545`

■ **Chain ID:** `31337`

■ **Símbolo:** `ETH`

- Feito isso, a carteira MetaMask se conectará ao simulador local exibindo o saldo de 10.000 ETH da conta importada. Ao rodar o frontend da aplicação (`npm run frontend:dev` na porta `3000`), a página reconhece e conecta-se via MetaMask exibindo o endereço da conta e o Chain ID `31337`.

4. Exemplos Práticos e Analogias

- **Analogia do Alias (Atalho):** Configurar um `alias` no terminal para abrir o navegador em modo debug (`google-chrome --remote-debugging-port=9222`) é como programar a discagem rápida do telefone celular. Em vez de digitar manualmente um número longo e difícil de lembrar, você apenas clica em um atalho curto de teclado.
- **A Simulação de Voo do Hardhat:** O Hardhat funciona como um simulador de voo para pilotos de avião. O desenvolvedor não deve fazer seus testes de contratos e transações diretamente na rede principal (Mainnet) com dinheiro real. O simulador provê um painel de controle (nó local) com combustível infinito e sem riscos (ETH fake de teste) para que tudo seja validado com segurança.
- **Importando Identidades:** A chave privada atua como uma assinatura digital. Ao copiar a chave privada fornecida pelo Hardhat e colá-la na MetaMask, o programador está importando a "identidade digital" daquela conta rica do simulador para dentro do seu gerenciador pessoal de carteiras.

5. Resumo dos Pontos Críticos (O que vai cair na prova)

- A necessidade de um ambiente controlado para blockchain, diferenciando os benefícios de usar o **Hardhat** (simulação local e gratuita) frente às desvantagens de testar diretamente na rede principal (Mainnet com taxas reais).
- O processo e propriedades exigidas para conexão remota de console: uso de `--remote-debugging-port=9222` e a configuração de attach na porta `9222` no `launch.json`.
- Os parâmetros obrigatórios e exatos para configurar a rede local do Hardhat na MetaMask: RPC URL `http://127.0.0.1:8545` e Chain ID `31337`.
- O papel da **Secret Recovery Phrase** (12 palavras) na criptografia assimétrica, servindo de

base para a geração e resgate seguro das chaves privadas.

- A finalidade do arquivo `hardhat.config.js` na definição da versão do Solidity (`0.8.20`) e das redes de testes.

? 6. Questões de Autoavaliação (Estilo Flashcard)

- **Pergunta 1: Qual a diferença entre a senha local definida no MetaMask e a Seed Phrase (Secret Recovery Phrase) de 12 palavras no que tange à recuperação da conta?**

Ver Resposta A senha local protege os dados da extensão apenas no dispositivo atual em que ela foi criada, funcionando como uma fechadura de acesso local. A Seed Phrase de 12 palavras é a raiz criptográfica que gera todas as chaves privadas; com ela, qualquer indivíduo consegue recuperar a carteira e ter controle total dos ativos em qualquer outro computador, mesmo que a senha local seja perdida.

- **Pergunta 2: Por que é necessário iniciar o Google Chrome através de linha de comando com o parâmetro `--remote-debugging-port=9222` para efetuar a depuração integrada com o VS Code?**

Ver Resposta Por padrão, os navegadores rodam de forma isolada por motivos de segurança. A flag `--remote-debugging-port=9222` instrui o Chrome a abrir uma porta de rede dedicada (9222) para que ferramentas externas (como o depurador do VS Code configurado no `launch.json`) possam se acoplar a ela, escutar eventos do console e depurar erros de JavaScript diretamente da IDE.

- **Pergunta 3: Ao rodar o comando `npm run hardhat node` no console, o que o Hardhat realiza em segundo plano e de onde vêm os saldos das carteiras exibidas?**

Ver Resposta O comando inicializa um nó de blockchain local emulado (rede Ethereum de simulação) na porta local 8545. Ele cria na memória da máquina 20 contas com chaves privadas associadas e credita em cada uma delas um saldo fictício inicial de 10.000 ETH para permitir o desenvolvimento e testes de transações locais sem gastar fundos reais.

- **Pergunta 4: O que é o Chain ID e qual o seu valor padrão para a rede de desenvolvimento local simulada pelo Hardhat?**

Ver Resposta O Chain ID é um identificador numérico exclusivo associado a cada rede blockchain. Ele é utilizado criptograficamente para assinar transações, garantindo que as assinaturas sejam válidas apenas na rede especificada e prevenindo ataques de replicação em outras blockchains. O Chain ID padrão da rede local do Hardhat é `31337`.

- **Pergunta 5: Explique como a MetaMask sabe que a conta importada possui 10.000 ETH se a rede é local e fictícia.**

Ver Resposta A MetaMask descobre o saldo consultando o nó de blockchain ao qual está conectada. Ao configurar a rede personalizada apontando para a URL local

(`http://127.0.0.1:8545`), a MetaMask envia requisições RPC para o nó local do Hardhat. Como o Hardhat mantém em sua memória a conta importada (via chave privada correspondente) com o saldo inicial fictício de 10.000 ETH, ele responde a requisição e a MetaMask exibe o valor em sua interface.

Aula 9

Disciplina: Blockchain e Web 3.0 **Vídeo de Referência:** @[aulas/videos/Aula 9 - Introdução à Web 3.0 (Parte 3).mp4]

✈ 1. Visão Geral e Objetivos

- **Contexto:** Esta aula aborda a integração programática entre uma aplicação frontend e a carteira digital MetaMask utilizando a API do provedor Ethereum (`window.ethereum`). O foco central é automatizar o fluxo de conexão do usuário, a detecção de redes e a troca automática para uma blockchain local de desenvolvimento (Hardhat), garantindo a persistência das configurações do perfil do navegador.
- **Objetivos de Aprendizagem:** O aluno deve ser capaz de configurar o perfil de desenvolvimento do Chrome para manter a persistência de dados, utilizar a API de provedor `window.ethereum` para interagir com a MetaMask por código JavaScript, implementar o chaveamento automático de redes usando `wallet_switchEthereumChain` e `wallet_addEthereumChain` com formatação hexadecimal para o Chain ID, e compreender a arquitetura geral de chamadas de métodos a contratos inteligentes.

🔑 2. Conceitos-Chave e Definições

- **`window.ethereum` (Provedor Ethereum):** Objeto injetado no escopo global do navegador por carteiras digitais (como a MetaMask) que atua como API/ponte para que o frontend envie requisições RPC à blockchain e interaja com a carteira.
- **`wallet_switchEthereumChain`:** Método RPC do provedor Ethereum utilizado para solicitar à carteira do usuário a troca ativa para uma rede blockchain específica, identificada por seu Chain ID em formato hexadecimal.
- **`wallet_addEthereumChain`:** Método RPC utilizado para registrar e configurar uma nova blockchain na carteira do usuário (informando RPC URL, Chain ID, símbolo da moeda e nome da rede) caso ela ainda não esteja cadastrada.
- **Hexadecimal no Chain ID:** Formato numérico em base 16 exigido pelas requisições RPC do padrão Ethereum para o identificador da rede (ex: o identificador decimal `31337` é passado como `0x7a69`).
- **Query Parameter (Query String):** Parâmetros de busca anexados ao final de uma URL (iniciando com `?` e separados por `&`) usados para passar dados de controle ao frontend (ex: `?showHardNetwork=1`).
- **Browser-Sync:** Ferramenta de desenvolvimento que cria um servidor local para arquivos estáticos (HTML/CSS/JS) e realiza o recarregamento automático (live reload) do navegador sempre que um arquivo do projeto é alterado.
- **hidden Attribute:** Atributo nativo do HTML usado para esconder elementos da interface gráfica. A manipulação de sua presença via JavaScript permite alternar dinamicamente a visibilidade de

componentes.

- **ABI (Application Binary Interface):** Arquivo JSON gerado na compilação de um contrato inteligente que descreve todas as suas funções, parâmetros e retornos, servindo de mapa para o frontend saber como invocar as funções do contrato.

3. Explicação Detalhada por Tópicos

- **Correção e Persistência do Perfil de Desenvolvimento do Chrome:**

- O professor inicia revisando o script de depuração remota do Chrome (`--remote-debugging-port=9222`).
- Ele aponta que apontar o `--user-data-dir` para a pasta temporária (`/tmp` ou similar) faz com que o sistema operacional limpe as chaves e dados da MetaMask entre os reinícios do computador.
- A correção sugerida é remapear a pasta do perfil de usuário para uma pasta fixa dentro do próprio diretório do projeto (como uma pasta local não-temporária), assegurando a persistência das carteiras importadas e configurações de rede de uma aula para outra.

- **Interface e Manipulação Dinâmica de Elementos HTML via Query Parameters:**

- A aplicação frontend possui uma seção oculta marcada com o ID `session site network` e o atributo `hidden`.
- Em `app.js` (no escopo do frontend), o código inspeciona os parâmetros da URL (`window.location.search`).
- Se o parâmetro `showHardNetwork=1` for detectado na URL, o JavaScript remove dinamicamente o atributo `hidden` da seção, expondo os controles adicionais de rede local na tela. O professor reforça que a manipulação de DOM em HTML/JavaScript é um pré-requisito e não o foco principal da disciplina.

- **Conexão e Requisições para Provedores Ethereum via API (`window.ethereum`):**

- O frontend se comunica com a MetaMask utilizando o objeto global `window.ethereum`.
- Toda ação (conectar carteira, trocar rede, assinar transação) é disparada via `ethereum.request({ method: '...' })`.
- Ao clicar no botão **Connect Wallet**, a aplicação dispara uma solicitação para conectar as contas e ler o endereço público do usuário (`eth_requestAccounts`) e o identificador da rede (`eth_chainId`).

- **Chaveamento e Cadastro Automático de Redes no MetaMask via Programação:**

- Em vez de exigir que o usuário adicione a rede local manualmente nas configurações da MetaMask, a aplicação automatiza o fluxo:
 1. Tenta trocar para a rede local do Hardhat chamando o método `wallet_switchEthereumChain` com o Chain ID em hexadecimal (`0x7a69`).
 2. Se a rede não estiver cadastrada na MetaMask (gerando um erro com o código

4902), o bloco `catch` intercepta o erro e chama o método

`wallet_addEthereumChain`.

3. Esse método exibe um modal amigável na MetaMask solicitando que o usuário aprove o cadastro da nova rede local com as seguintes configurações passadas no objeto:

- **Chain ID:** `0x7a69` (decimal `31337`)
- **Network Name:** local host web3 (ou similar)
- **RPC URL:** `http://127.0.0.1:8545`
- **Native Currency:** Nome `Ether`, Símbolo `ETH`, Decimais `18`

4. O usuário confirma a aprovação e a MetaMask chaveia a rede ativa automaticamente.

- **Visão Geral sobre Chamadas de Funções a Contratos Inteligentes:**

- O professor antecipa o funcionamento da aplicação completa "Hello World".
- Um contrato inteligente simples compilado e implantado na rede local possui a função `getMessage()` que retorna a string `"hello from blockchain"`.
- No frontend, a execução de `contract.getMessage()` executa uma chamada de leitura rápida na blockchain e exibe o retorno da string na tela ao clicar no botão **Get Message**.
- É destacado que, na próxima aula, será detalhada a lógica profunda de como essa chamada trafega via requisições JSON-RPC, codificação ABI e rede sob o capô, indo além de simples cliques em botões.

4. Exemplos Práticos e Analogias

- **Analogia da Conversão Decimal para Hexadecimal (`31337` $\rightarrow$ `0x7a69`):** É equivalente a enviar uma carta internacional. Embora você entenda o número do seu CEP de forma decimal, a central postal exige um formato padronizado internacional em código de barras para triagem eletrônica rápida. Para a MetaMask, o Chain ID numérico deve ser empacotado em notação hexadecimal.
- **O Erro do Perfil Temporário (TMP):** Salvar as configurações da MetaMask na pasta `/tmp` é como construir uma barraca de acampamento em uma praia de maré alta. Funciona perfeitamente durante a tarde (enquanto você desenvolve), mas na manhã seguinte (ao reiniciar a máquina), a maré sobe (o sistema operacional roda scripts de limpeza) e arrasta toda a sua estrutura embora.
- **O Fluxo de `wallet_switch` e `wallet_add`:** Imagine tentar sintonizar uma rádio local no carro. O `wallet_switch` é o botão de memória do rádio. Se a estação local do Hardhat já estiver programada, ela sintoniza na hora. Caso contrário, o rádio busca a frequência e exibe uma mensagem na tela perguntando se você quer salvar a nova estação (`wallet_add`) para poder ouvi-la.

5. Resumo dos Pontos Críticos (O que vai cair na prova)

- A utilização do objeto `window.ethereum` como o provedor e canal principal de dados entre aplicações Web 3.0 e a blockchain.
- A formatação obrigatória em **hexadecimal** para parâmetros de Chain ID em chamadas RPC da MetaMask (ex: `0x7a69` para o Chain ID local do Hardhat `31337`).
- O fluxo condicional de chaveamento de rede: a tentativa inicial via `wallet_switchEthereumChain` seguida de `wallet_addEthereumChain` em caso de falha por ausência da rede.
- O impacto do remapeamento do `--user-data-dir` do Chrome para garantir a retenção estável de dados e evitar o reset constante das chaves privadas da carteira MetaMask.
- A estrutura de uma chamada a um contrato inteligente no frontend (`contract.functionName()`) e sua dependência do mapeamento de funções.

? 6. Questões de Autoavaliação (Estilo Flashcard)

- **Pergunta 1: Por que a MetaMask exige que o Chain ID nas chamadas de troca ou adição de rede via programação seja especificado em hexadecimal (como `0x7a69`) ao invés do decimal tradicional `31337`?**

Ver Resposta Porque a API de comunicação interna das redes baseadas em Ethereum (especificação EIP-1474 e EIP-3085) adota a codificação JSON-RPC padrão, a qual exige que valores numéricos como Chain IDs e quantidades de gas sejam enviados codificados como strings em representação hexadecimal prefixadas por ``0x`` para padronização e eficiência de parsing no protocolo de rede.

- **Pergunta 2: Descreva a sequência lógica de tratamento de erro que permite que um site cadastre e mude a rede do usuário para o nó do Hardhat sem que ele faça isso manualmente no menu da carteira.**

Ver Resposta O código Javascript do frontend executa um bloco ``try-catch`` chamando ``wallet_switchEthereumChain`` com o Chain ID ``0x7a69``. Se a rede local não estiver cadastrada na MetaMask do usuário, a chamada falha e lança um erro com um código de status específico (tipicamente o código ``4902``). O bloco ``catch`` intercepta esse erro de "rede não encontrada" e executa a chamada ao método ``wallet_addEthereumChain`` passando as informações de configuração da rede (URL RPC e metadados) para que a MetaMask exiba o modal de adição de rede ao usuário.

- **Pergunta 3: Qual a função do parâmetro `--remote-debugging-port` e como ele atua em conjunto com o VS Code para melhorar a produtividade do desenvolvedor?**

Ver Resposta Este parâmetro expõe uma porta de comunicação em rede na instância aberta do Google Chrome. O VS Code conecta-se a essa porta através de um perfil de "anexação" (attach) no arquivo ``launch.json``. Uma vez conectado, ele cria um canal bidirecional onde erros do console da web, mensagens e logs do navegador são replicados em tempo real diretamente dentro do painel Debug Console do próprio editor de código, eliminando a necessidade de ficar alternando entre o código e as ferramentas de desenvolvedor do navegador.

- **Pergunta 4: O que é e qual o papel do objeto `window.ethereum` no desenvolvimento de aplicações frontend da Web 3.0?**

Ver Resposta O `window.ethereum` é o objeto do provedor de API que a extensão da MetaMask injeta dinamicamente na página web. Ele atua como o intermediário principal entre o código JavaScript do frontend e a rede blockchain, permitindo que a aplicação faça requisições RPC para ler contas, assinar transações, escutar eventos de mudança de carteira ou rede, e interagir com contratos inteligentes.

- **Pergunta 5: Por que o uso de diretórios temporários (como `/tmp`) no parâmetro `--user-data-dir` do Chrome é desencorajado para desenvolvimento recorrente e como isso afeta a MetaMask?**

Ver Resposta Porque o diretório temporário é apagado rotineiramente pelo sistema operacional. Quando isso ocorre, todo o perfil de configurações do Chrome associado é limpo, resultando na perda das informações instaladas na extensão da MetaMask (incluindo as chaves privadas importadas das contas locais do Hardhat, redes customizadas adicionadas e a Seed Phrase da carteira), exigindo que o desenvolvedor refaça todo o setup do zero a cada reinício.

Aula 10

Disciplina: Blockchain e Web 3.0 **Vídeo de Referência:** @[aulas/videos/Aula 10 - Introdução à Web 3.0 (Parte 4).mp4]

🔑 1. Visão Geral e Objetivos

- **Contexto:** Esta aula é dedicada à compilação, implantação (deploy) e interação de contratos inteligentes no ambiente simulado do Hardhat. O foco principal é entender como o código em Solidity é convertido em bytecode executável pela EVM, como a aplicação frontend se conecta a um contrato implantado e a diferença técnica de custos entre transações de escrita e consultas gratuitas.
- **Objetivos de Aprendizagem:** O aluno deve compreender o processo de compilação em Solidity, entender os conceitos de Bytecode, Opcodes e versões de EVM (como a versão Paris), programar e executar scripts de deploy locais, associar o endereço do contrato no frontend e explicar a utilidade e gratuidade das funções puras (**pure**) de leitura na blockchain, bem como a arquitetura de armazenamento externo via IPFS.

🔑 2. Conceitos-Chave e Definições

- **Bytecode:** Código binário de baixo nível gerado após a compilação do contrato Solidity que pode ser diretamente interpretado e executado pela EVM.
- **Opcodes (Operation Codes):** Instruções elementares de máquina virtual baseadas na pilha da EVM (ex: **ADD 0x01**, **MUL**, **SSTORE**) que executam ações computacionais fundamentais.
- **EVM Target / Versão da EVM:** A versão específica de regras e opcodes da máquina virtual para a qual o contrato é compilado (ex: a versão **Paris** correspondente ao The Merge do Ethereum, ou Cancun, Shanghai, etc.).
- **Deploy (Implantação):** O processo de enviar o bytecode de um contrato inteligente para a rede blockchain através de uma transação, criando um endereço de contrato único e persistente.
- **Contract Address (Endereço do Contrato):** Identificador alfanumérico exclusivo na blockchain que localiza o contrato inteligente implantado e serve como destino para chamadas e transações.
- **pure Keyword:** Modificador de mutabilidade de estado em Solidity que declara que a função não lê e nem modifica o estado da blockchain, tornando sua execução off-chain inteiramente gratuita.
- **eth_call:** Requisição RPC de leitura imediata enviada a um nó da blockchain que executa o código de uma função localmente sem criar uma transação no livro-razão e sem custo de gas.
- **Base64 / Armazenamento de Arquivos em Blockchain:** Representação binária em string que permite embutir arquivos no código, mas cujo armazenamento direto em contratos inteligentes é desencorajado devido ao elevado custo computacional.
- **IPFS (InterPlanetary File System):** Protocolo descentralizado de rede ponto a ponto (P2P)

projetado para armazenar e compartilhar arquivos de forma distribuída, gerando um hash único de conteúdo imutável.

- **EIP (Ethereum Improvement Proposal):** Documentos formais de propostas de melhoria técnicas e padrões adotados pela comunidade Ethereum para evoluir a máquina virtual e as regras da rede.

3. Explicação Detalhada por Tópicos

- **Compilação e a Anatomia Interna da EVM:**

- O professor demonstra a compilação do contrato inteligente utilizando o comando `npm run compile` (mapeado para `hardhat compile`).
- Ele esclarece que a EVM (Ethereum Virtual Machine) funciona baseada em uma arquitetura de pilha e executa instruções chamadas **Opcodes** (códigos de operação).
- Cada instrução de alto nível em Solidity (como uma soma) é convertida pelo compilador em uma série de Opcodes (ex: `ADD` correspondente ao byte `0x01`).
- As versões da EVM utilizam codinomes de cidades (Paris, Shanghai, Cancun, Prague). A versão adotada na aula é a **Paris** (de setembro de 2022, data do "The Merge"). Cada atualização de versão introduz novos opcodes e regras de consenso.

- **Declaração do Contrato "Hello World" em Solidity:**

- A estrutura básica de um contrato simulando uma classe é definida com a palavra-chave `contract`:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.20;

contract HelloWorld {
    function getMessage() public pure returns (string memory) {
        return unicode"hello from blockchain 🚀";
    }
}
```

- **Uso do modificador `unicode`:** O professor destaca que para strings que contêm caracteres especiais ou emojis (como o foguete 🚀), é obrigatório prefixar o literal com a palavra-chave `unicode` para evitar falhas de compilação causadas por codificação não-ASCII.

- **Fluxo de Deploy e Estrutura do Script de Implantação:**

- Para publicar o contrato na rede local, utiliza-se um script JS localizado em `scripts/deploy.js` que é invocado pelo comando `npm run deploy:local` (mapeado para `hardhat run scripts/deploy.js --network localhost`).
- O deploy consiste em empacotar o bytecode do contrato em uma transação assinada e enviá-la para a rede local (`http://127.0.0.1:8545`).

- Como o deploy altera o estado global da rede (adicionando um novo contrato), ele é classificado como uma transação que consome saldo da carteira do implantador para pagar a taxa de gas. Ao final da execução, o terminal do Hardhat exibe as chamadas RPC efetuadas e retorna o **Contract Address** gerado.

- **Conexão Frontend e Invocação RPC:**

- Após obter o endereço correto do contrato, o desenvolvedor atualiza a variável `contractAddress` no script do frontend.
- Com a MetaMask conectada na rede local do Hardhat (`Chain ID 31337`), o usuário clica em **Get Message**.
- A aplicação frontend realiza uma requisição de leitura disparando o método `eth_call` na blockchain.
- O terminal do Hardhat registra a requisição:

- **Origem:** Endereço da carteira importada (Account #0 - final `92266`).

- **Destino:** Endereço do contrato HelloWorld (final `80a3`).

- **Função invocada:** `getMessage()`.

- O texto de retorno `"hello from blockchain 🦊"` é impresso na interface da página.

- **O Modificador `pure` e a Gratuidade de Leitura:**

- O modificador `pure` sinaliza à EVM que a função não acessa variáveis globais de estado e nem grava informações na blockchain.
- Como a chamada é estritamente computacional e não modifica o estado compartilhado da blockchain, ela é processada inteiramente localmente pelo nó que recebe a requisição. Portanto, **não há geração de transação minerada e o custo de gas é zero** (gratuito) para o usuário quando chamada off-chain.

- **Arquitetura de Armazenamento de Grande Volume e IPFS:**

- Respondendo a questionamentos dos alunos, o professor explica que colocar grandes volumes de dados (como imagens raw ou sequências longas em Base64) diretamente em variáveis de contratos inteligentes na blockchain é inviável e extremamente caro, pois todos os validadores da rede precisariam replicar e armazenar esses bytes.
- A solução arquitetural padrão na Web3 (como a usada em NFTs) consiste em hospedar os arquivos pesados de forma descentralizada no **IPFS (InterPlanetary File System)**.
- O IPFS armazena o arquivo em uma rede P2P distribuída e gera uma referência criptográfica estável (hash). O contrato inteligente armazena na blockchain apenas essa referência/endereço curto, mantendo o custo de gas mínimo e preservando a descentralização.

4. Exemplos Práticos e Analogias

- **Analogia da VM do Solidity e a JVM:** A EVM funciona de modo muito similar à JVM (Java

Virtual Machine) ou ao interpretador de Python. Você escreve código em alto nível (Java ou Solidity), o compilador traduz isso para um formato intermediário estruturado em opcodes de pilha (Bytecode), e a máquina virtual interpreta esses opcodes executando no processador.

- **Analogia do IPFS e o Guarda-Volumes do Aeroporto:** Armazenar uma imagem na blockchain é como tentar viajar carregando um armário de madeira gigante nas mãos. Você pagará excesso de bagagem caríssimo a cada embarque. O padrão IPFS funciona como deixar o armário em um guarda-volumes do aeroporto (rede IPFS) e viajar carregando apenas o bilhete impresso de retirada (hash/endereço). O bilhete é leve e barato de guardar na sua carteira (blockchain), mas dá acesso garantido ao armário.
- **O Custo Zero do Modificador `pure`:** Consultar uma função `pure` na blockchain é como ler uma página de um livro na biblioteca pública. Você consome energia física (processamento local no nó) mas não altera o conteúdo do livro, sendo totalmente de graça. Gravar dados (funções que mudam o estado) equivale a escrever uma página nova no livro oficial de registros da cidade; exige taxas cartoriais (gas) e o carimbo de múltiplos tabeliões (consenso).

🎯 5. Resumo dos Pontos Críticos (O que vai cair na prova)

- O processo de compilação em blockchain, conceituando **Bytecode**, **Opcodes** (comandos elementares) e a lógica de execução sob pilha da EVM.
- As versões de EVM e a importância histórica da versão **Paris**, que introduziu mudanças chave no algoritmo de consenso e novos opcodes.
- O fluxo operacional de deploy de contratos, sendo reconhecido como uma transação que demanda pagamento de taxas de gas e gera um **Contract Address**.
- A definição e a vantagem econômica do modificador `pure`: ausência de interação com estado de armazenamento e gratuidade em chamadas off-chain via `eth_call`.
- O padrão arquitetural de armazenamento híbrido da Web3: dados estruturados leves e apontadores na blockchain, e arquivos pesados no **IPFS**.

? 6. Questões de Autoavaliação (Estilo Flashcard)

- **Pergunta 1: Por que é necessário utilizar o modificador `unicode` antes de uma string literal com emojis em Solidity (ex: `unicode"hello 🦉"`) e o que ocorre se ele for omitido?**

Ver Resposta O Solidity por padrão interpreta strings literais simples utilizando codificação estritamente ASCII. Caracteres fora da tabela ASCII tradicional (como emojis e acentos complexos de caracteres Unicode) contêm sequências de múltiplos bytes que o compilador não reconhece de forma nativa nessa codificação. Se o modificador `unicode`` for omitido, o compilador Solidity falhará na análise sintática e emitirá um erro de compilação de caractere inválido.

- **Pergunta 2: O que são Opcodes no contexto da EVM e como uma operação simples de adição escrita em Solidity é convertida para ser rodada pela rede?**

Ver Resposta Opcodes são os códigos binários de instrução de operação em baixo nível

interpretados pela EVM (que opera baseada em pilhas). Quando uma linha de código Solidity (como ``1 + 1``) é compilada, o compilador converte os operadores matemáticos no opcode correspondente (como o opcode ``ADD`` de valor hexadecimal ``0x01``), empilhando os argumentos e executando a soma a nível de máquina virtual.

- **Pergunta 3: Qual a diferença técnica entre realizar uma requisição de leitura `eth_call` em uma função `pure` e uma transação de escrita (deploy ou alteração de estado) em relação ao consenso da rede e taxas de gas?**

Ver Resposta Uma requisição ``eth_call`` de leitura a uma função ``pure`` é processada localmente e de forma isolada pelo nó RPC que recebeu a chamada, sem necessidade de consenso de rede global ou registro no bloco, o que a torna instantânea e totalmente gratuita (sem gas). Já uma transação de escrita modifica o estado global da blockchain, exigindo que a instrução seja propagada para toda a rede, validada via algoritmo de consenso e persistida por todos os nós em um novo bloco, o que demanda tempo e o pagamento obrigatório de taxas de gas para remunerar os validadores pelo processamento.

- **Pergunta 4: Explique por que não se deve armazenar grandes volumes de dados (como imagens e mídias brutas) diretamente nos contratos inteligentes da blockchain e descreva a solução descentralizada padrão adotada para contornar isso.**

Ver Resposta Armazenar grandes volumes de dados diretamente na blockchain é extremamente caro e ineficiente, pois cada nó validador na rede é forçado a baixar, processar e guardar perpetuamente esses dados em seus discos rígidos locais, gerando taxas de gas inviáveis para a transação. A solução padrão é salvar a mídia em um sistema descentralizado P2P como o IPFS (InterPlanetary File System), obter o link/hash curto de conteúdo e armazenar apenas esse apontador leve no contrato inteligente, mantendo a transação barata sem abrir mão da imutabilidade e distribuição.

- **Pergunta 5: O que representa a versão "Paris" da EVM exibida durante a compilação de contratos inteligentes e qual seu significado histórico para o Ethereum?**

Ver Resposta A versão "Paris" representa a especificação e o conjunto de regras computacionais (incluindo novos opcodes e comportamento da EVM) introduzidos na atualização histórica do Ethereum conhecida como "The Merge", em setembro de 2022. Essa atualização marcou a transição da rede Ethereum do algoritmo de consenso Proof of Work (Mineração física) para o Proof of Stake (Validação por participação), unificando a camada de execução e alterando profundamente o funcionamento e segurança da rede.

Aula 11

Disciplina: Blockchain e Web 3.0 **Vídeo de Referência:** @[aulas/videos/Aula 11 - Introdução à Web 3.0 (Parte 5).mp4]

🔗 1. Visão Geral e Objetivos

- **Contexto:** Esta aula aprofunda a integração prática entre contratos inteligentes e aplicações frontend. O foco principal é compreender as diferenças entre modificadores de visibilidade e estado (**pure** e **view**), analisar a estrutura e utilidade da ABI (Application Binary Interface) gerada na compilação, e entender como a aplicação web se reconecta programaticamente a novas instâncias de contratos locais do Hardhat após a reinicialização da rede.
- **Objetivos de Aprendizagem:** O aluno deve compreender e diferenciar conceitualmente e na prática as funções puras (**pure**) e funções de visualização (**view**), interpretar a estrutura interna de um arquivo de artefato de compilação JSON contendo a ABI e o Bytecode, integrar a ABI atualizada no frontend via JavaScript e entender os fluxos de atualização manual versus dinâmica em projetos Web3 profissionais.

🔑 2. Conceitos-Chave e Definições

- **Variáveis de Estado (Storage Variables):** Variáveis declaradas no escopo global do contrato inteligente cujos valores são gravados e mantidos permanentemente no livro-razão (armazenamento persistente) da blockchain.
- **view Keyword:** Modificador de mutabilidade em Solidity usado para sinalizar que a função realiza leitura de variáveis de estado do contrato, mas não modifica o estado da blockchain, sendo gratuita quando executada off-chain.
- **pure Keyword:** Modificador de mutabilidade em Solidity usado para sinalizar que a função não lê dados de estado e nem escreve na blockchain, operando apenas com dados fornecidos no próprio escopo ou parâmetros.
- **ABI (Application Binary Interface):** Estrutura padronizada em formato JSON que descreve os métodos, parâmetros de entrada, tipos de retorno e estados de mutabilidade de um contrato inteligente, permitindo que o frontend interaja com o bytecode do contrato.
- **ethers.Contract:** Classe provida pela biblioteca **ethers.js** que instancia um objeto JavaScript representando o contrato inteligente a partir do seu endereço de deploy, sua ABI e o assinante (carteira do usuário).
- **Bytecode de Contrato:** Código binário executável implantado de forma definitiva em um endereço da blockchain, traduzido de Solidity para opcodes reconhecidos pela EVM.
- **nonpayable:** Estado padrão de mutabilidade de funções que podem alterar o estado das variáveis da blockchain, mas que não recebem ou transferem Ether nativo durante sua chamada.
- **Hardcode (Codificação Estática):** Prática de inserir dados ou configurações diretamente no

código-fonte do programa de forma fixa, dificultando a manutenção automática.

- **Carregamento Dinâmico de ABI:** Técnica profissional em que a aplicação frontend consome a ABI lendo diretamente um arquivo `.json` exportado da compilação do contrato, evitando a necessidade de copiar e colar código manualmente.

3. Explicação Detalhada por Tópicos

- **Persistência de Rede e Atualização de Endereços de Deploy:**

- O professor demonstra que, ao reiniciar a blockchain local do Hardhat (`npm run chain`), todos os dados anteriores são limpos e o contrato precisa ser implantado novamente (`npm run deploy:local`).
- Cada nova implantação gera um **Contract Address** diferente. O desenvolvedor deve atualizar manualmente a variável `contractAddress` no arquivo `[app.js](file:///Ubuntu/home/victorhugo/cc-blockchain-e-web-3-2026.1/aulas/pratica/aula-1/frontend/app.js)` para que a aplicação web se reconecte corretamente.

- **Mutabilidade do Estado: `pure` vs `view` em Solidity:**

- O contrato `HelloWorld` é estendido com variáveis e novas funções:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.20;

contract HelloWorld {
    uint256 public storedData;

    // Modifica o estado do contrato (gasta gas se chamado por outra transação on-chain)
    function set(uint256 x) public {
        storedData = x;
    }

    // Apenas lê variáveis de estado (view) - gratuito off-chain
    function get() public view returns (uint256) {
        return storedData;
    }

    // Não lê nem modifica o estado (pure) - gratuito off-chain
    function getMessage() public pure returns (string memory) {
        return unicode"hello from blockchain 🦉";
    }
}
```

- **Funções `pure`:** Utilizadas quando o retorno não depende de nenhuma variável fora da própria função (ex: retornar um valor estático ou realizar cálculos matemáticos puramente com parâmetros de entrada).
- **Funções `view`:** Utilizadas quando a função lê uma variável persistente global do contrato

(como `storedData`), mas garante que não a modificará.

- **Custo computacional:** Nenhuma das duas funções consome gas (taxas de transação) quando chamadas de fora da blockchain (por um frontend local via `eth_call`). O compilador informa essa mutabilidade à EVM para otimizar a velocidade de processamento do nó local.

• Anatomia da ABI (Application Binary Interface):

- Toda compilação gera um arquivo de artefato JSON (ex: `HelloWorld.json`). Ele contém a **ABI**, que descreve os métodos para a aplicação frontend.
- O professor analisa os elementos estruturais da ABI no JSON:
 - **get:** Nome da função é `get`, parâmetros de entrada (`inputs`) é um array vazio `[]`, parâmetros de saída (`outputs`) contém um tipo `uint256`, e a mutabilidade é classificada como `view`.
 - **set:** Nome da função é `set`, `inputs` contém o parâmetro `x` do tipo `uint256`, `outputs` é um array vazio `[]`, e a mutabilidade de estado é sinalizada como **`nonpayable`** (altera o estado, mas não realiza transações financeiras de Ether).
 - **getMessage:** Tipo `function`, `inputs` vazio `[]`, `outputs` do tipo `string`, e mutabilidade classificada como `pure`.

• Integração Frontend com a Classe `ethers.Contract`:

- A biblioteca `ethers.js` é usada para criar uma ponte no frontend JavaScript:

```
const contract = new ethers.Contract(contractAddress, contractABI, signer);
```

- Para invocar o contrato, o JavaScript apenas executa a chamada na instância: `const message = await contract.getMessage();`
- O `ethers.js` consulta a ABI para codificar a chamada no formato exigido pela EVM e despacha a requisição HTTP JSON-RPC ao nó do Hardhat.
- O professor demonstra no console do Hardhat o log gerado ao clicar no botão: `eth_call` apontando da carteira do usuário para o endereço do contrato e chamando a função `getMessage()`.
- O resultado retornado é atribuído ao HTML do DOM usando propriedades como `innerText` no elemento com ID `message`.

• Sincronização de ABI: Fluxo Manual vs Profissional:

- **Fluxo Didático (Manual):** O desenvolvedor compila o contrato, abre o arquivo `.json` gerado na pasta `artifacts`, copia o array correspondente ao campo `abi` e cola diretamente na declaração de variável no frontend (`app.js`).
- **Fluxo Profissional (Dinâmico):** Em projetos reais, o código do contrato inteligente e o do frontend residem em repositórios separados. A equipe de contratos fornece o arquivo `.json` compilado para o time de frontend. O código da aplicação frontend é programado

para fazer um carregamento dinâmico do JSON (via chamadas `fetch` ou instruções `require`), eliminando a necessidade de reescrever ou copiar código em variáveis estáticas todas as vezes que o contrato mudar.

4. Exemplos Práticos e Analogias

- **Analogia da ABI e o Manual do Eletrodoméstico:** A ABI é como o manual de instruções ou a disposição dos botões físicos de um eletrodoméstico (por exemplo, um micro-ondas). A aplicação frontend é o usuário que quer usar o micro-ondas. Ela não precisa entender os circuitos integrados internos e nem as placas eletrônicas (o Bytecode), mas precisa do painel de botões e do manual (a ABI) que diz exatamente qual botão apertar (nome do método) e o que colocar no prato (parâmetros de entrada) para obter o alimento aquecido (retorno).
- **Analogia de `pure` vs `view` e a Lousa da Sala:** Imagine a blockchain como a lousa de uma sala de aula.
 - Uma função `view` é como um aluno que olha para a lousa e lê em voz alta o que o professor escreveu (lê a variável de estado `storedData`).
 - Uma função `pure` é como um aluno que realiza uma conta matemática em sua própria cabeça, sem sequer olhar para a lousa (não lê e nem escreve na blockchain).
 - Ambas as ações não alteram o que está escrito no quadro e, portanto, não gastam "giz" (gas).

5. Resumo dos Pontos Críticos (O que vai cair na prova)

- A diferença fundamental entre os modificadores de mutabilidade: `pure` (não acessa o estado e nem o lê) e `view` (acessa e lê o estado global, mas não o altera).
- A gratuidade de gas das funções `pure` e `view` quando invocadas por um cliente externo (fora de uma transação on-chain) por meio do comando `eth_call`.
- O papel da **ABI (Application Binary Interface)** como o mapeador estrutural necessário para traduzir requisições JavaScript em chamadas binárias de opcodes da EVM.
- A estrutura da classe `ethers.Contract` no frontend, exigindo os três parâmetros obrigatórios: endereço do contrato, ABI e assinante/provedor (`signer`).
- O conceito do estado mutável `nonpayable` na ABI, representando funções que alteram variáveis de estado, mas não aceitam ou operam moedas nativas (Ether).

? 6. Questões de Autoavaliação (Estilo Flashcard)

- **Pergunta 1: Se o desenvolvedor modificar uma função em seu contrato inteligente adicionando uma nova leitura de variável global, o que ele precisa fazer no frontend da aplicação para que essa mudança seja refletida com sucesso?**

Ver Resposta Ele deve compilar o contrato inteligente novamente para atualizar a ABI. Em seguida, deve atualizar a cópia da ABI no frontend (copiando o novo JSON da pasta ``artifacts``

ou recarregando o arquivo JSON dinamicamente), pois sem a nova definição da interface na ABI, o objeto `ethers.Contract` no frontend não reconhecerá a alteração da assinatura do método e a chamada falhará.

- **Pergunta 2: Por que uma função Solidity declarada com o modificador `view` não pode ser marcada com o modificador `pure`?**

Ver Resposta Porque o modificador `view` é utilizado especificamente quando a função necessita ler uma variável de estado persistente do contrato (como o valor de `storedData`). O modificador `pure` é muito mais restritivo e proíbe terminantemente tanto a alteração quanto qualquer leitura do estado da blockchain, permitindo apenas operações com variáveis locais à função ou parâmetros de entrada.

- **Pergunta 3: Qual a diferença de finalidade entre o Bytecode e a ABI contidos no arquivo JSON gerado na compilação do contrato inteligentes?**

Ver Resposta O Bytecode é a representação em baixo nível do contrato em formato hexadecimal que será de fato enviado e gravado na blockchain para execução direta da EVM. A ABI é uma interface estruturada em JSON que descreve os métodos, parâmetros e tipos de dados do contrato, servindo como um "mapa" para que linguagens externas (como o JavaScript no frontend) saibam como formatar chamadas e interagir com o Bytecode gravado.

- **Pergunta 4: O que significa a propriedade `stateMutability` na descrição de uma função na ABI e quais valores ela pode assumir?**

Ver Resposta A propriedade `stateMutability` especifica o nível de interação da função com o estado da blockchain. Ela indica à EVM o comportamento de leitura/escrita e pode assumir quatro valores principais: `pure` (sem ler nem alterar estado), `view` (apenas leitura do estado), `nonpayable` (pode alterar o estado, mas não recebe Ether) e `payable` (pode alterar o estado e aceita receber Ether nativo na transação).

- **Pergunta 5: Como a biblioteca `ethers.js` sabe o que enviar ao nó da blockchain quando chamamos `contract.getMessage()` no JavaScript?**

Ver Resposta A biblioteca `ethers.js` consulta o array da ABI do contrato para encontrar a função `getMessage`. Ela verifica o nome, os parâmetros (que nesse caso estão vazios) e o retorno. Com base nisso, ela gera uma chamada padronizada contendo a assinatura criptografada da função (seletor de função) e codifica o payload, empacotando-o em uma requisição HTTP JSON-RPC com o método `eth_call` que é enviada ao nó local da blockchain.

Aula 12

Disciplina: Blockchain e Web 3.0 **Vídeo de Referência:** @[aulas/videos/Aula 12 - Introdução à Web 3.0 (Parte 6).mp4]

🔑 1. Visão Geral e Objetivos

- **Contexto:** Esta aula aborda a manipulação prática de variáveis de estado na blockchain por meio de interações frontend com o contrato inteligente. São explicados os fluxos de desenvolvimento de escrita de dados, validação de entradas do usuário com expressões regulares, custos de transação (gas fees), o ciclo de mineração e confirmação de blocos (`tx.wait()`) e o princípio da atomicidade e reversão de transações em Solidity.
- **Objetivos de Aprendizagem:** O aluno deve compreender como acoplar e desacoplar variáveis de configuração em projetos Web3 (como o uso de `.env`), validar entradas numéricas no frontend usando Regex, assinar transações de alteração de estado na MetaMask, entender o papel das gas fees e dos nós validadores, monitorar a confirmação de blocos via código e explicar a propriedade de atomicidade e reversão em Solidity.

🔑 2. Conceitos-Chave e Definições

- **Desacoplamento de Configurações:** Prática de separar parâmetros mutáveis (como o endereço de deploy de um contrato) do código principal, utilizando variáveis de ambiente (arquivos `.env`) para facilitar deploys em diferentes infraestruturas (ex: contêineres Docker, Vercel).
- **Expressões Regulares (Regex):** Padrões de caracteres matemáticos usados para realizar buscas, substituições e validações de strings (ex: `/^\d+$/` para verificar se um texto contém apenas dígitos inteiros).
- **Gas Fee (Taxa de Rede):** Custo em Ether pago pelo usuário para executar transações que alteram o estado da blockchain, cuja finalidade é remunerar os validadores da rede pelo poder computacional gasto.
- **Nó Validador:** Computador participante do consenso da rede blockchain encarregado de agrupar transações em blocos, executar os bytecodes das transações e mantê-las registradas no histórico imutável.
- **Atomicidade:** Propriedade que garante que uma transação de blockchain seja executada por completo ou não seja executada de forma alguma. Se qualquer operação falhar, todo o estado anterior é restaurado.
- **Reversão (`revert`):** Interrupção abrupta na execução de uma função de contrato inteligente que desfaz todas as alterações de dados feitas na transação até aquele ponto, estornando o estado do ledger.
- **`require`:** Função nativa em Solidity usada para validar condições de execução. Se a condição especificada for falsa, a execução é interrompida imediatamente e ocorre a reversão da

transação, opcionalmente retornando uma mensagem de erro.

- **`tx.wait()`**: Método da biblioteca `ethers.js` que pausa a linha de execução do frontend até que a transação correspondente seja incluída em um bloco minerado e confirmada pela rede blockchain.
- **Chamada Externa de Contrato**: Capacidade que um contrato inteligente possui de invocar e executar funções de outro contrato inteligente já implantado na mesma blockchain.

3. Explicação Detalhada por Tópicos

- **Organização das Configurações do Projeto e Variáveis de Ambiente:**

- O professor discute as boas práticas de desenvolvimento de software no que diz respeito a configurações fixas no código (*hard-coded*).
- Em ambientes reais, o ideal é extrair chaves e endereços para um arquivo de variáveis de ambiente (`.env`) e ler via `process.env.CONTRACT_ADDRESS`. Isso permite que a aplicação mude de contrato sem precisar de novas compilações do código-fonte ou rebuilds de contêineres Docker/Vercel.
- Por ser um script frontend vanilla simples rodando diretamente no navegador, a importação direta de bibliotecas de Node (como `dotenv` via `require`) gera exceções de escopo. Por razões didáticas, o professor reverte o ajuste e mantém temporariamente o endereço fixo no `[app.js]`(`file:///Ubuntu/home/victorhugo/cc-blockchain-e-web-3-2026.1/aulas/pratica/aula-1/frontend/app.js`).

- **Validação de Inputs com Expressões Regulares (Regex):**

- Para evitar o envio de dados incorretos à blockchain, o frontend realiza a validação do campo de texto antes de disparar a transação.
- O professor aborda o conceito de **Expressões Regulares (Regex)**, demonstrando como testar se a entrada é um número inteiro válido usando o padrão `/^\d+$/`:
 - `^` indica o início da string.
 - `\d` representa qualquer caractere de dígito de 0 a 9.
 - `+` indica que o dígito deve se repetir uma ou mais vezes.
 - `$` indica o final da string.
- Se a entrada falhar no teste, a aplicação exibe um alerta e impede a chamada à blockchain. Exemplos complexos de Regex (como validações de e-mail e CPF) também são comentados.

- **Fluxo de Escrita em Variáveis de Estado (`set`):**

- Quando o usuário clica no botão **Set**, a aplicação chama a função `set(value)` do contrato.
- Diferente de consultas (`getMessage` ou `get`), a escrita de dados exige alteração na blockchain.
- A MetaMask abre uma janela de consentimento, informando que a transação consumirá

fundos e exibindo a taxa de gas cobrada (ex: 17 centavos de dólar convertidos em Ether).

- O professor explica a lógica da taxa: ela incentiva financeiramente os **nós validadores** a manterem a infraestrutura de processamento da rede.

- **Mineração, Confirmação e o método `tx.wait()`:**

- Uma vez que a transação é confirmada e assinada pelo usuário na MetaMask, ela é transmitida para a rede.
- O método do contrato retorna uma transação (`tx`). No JavaScript, chama-se:

```
await tx.wait();
```

- Esse comando bloqueia a execução do JavaScript até que a transação seja minerada e colocada em um bloco. Após a confirmação, o frontend chama a função `get()` do contrato para obter a variável atualizada e exibir a mensagem de sucesso na interface do usuário.

- **Princípio de Atomicidade e Reversão em Solidity:**

- Durante os testes de debug, o console exibe a mensagem de erro `Transaction reverted without a reason`. Mesmo com a mensagem, o valor do contrato é alterado. O professor explica que isso ocorre por ruídos do ambiente local e usa a oportunidade para detalhar o funcionamento computacional da **reversão**.
- Em Solidity, a execução de transações é **atômica**:
 1. Se uma função possui múltiplos passos (ex: transferir fundos, salvar registro local, chamar contrato secundário).
 2. Se os dois primeiros passos executam com sucesso, mas o terceiro passo falha (reverte ou gera erro).
 3. A EVM reverte a transação inteira: todas as transferências e alterações de variáveis dos passos 1 e 2 são desfeitas na memória global, retornando o estado anterior ao início da chamada.
- A validação de pré-condições é feita usando a instrução `require(condicao, "erro")`. Se a condição for falsa, o processamento para e desfaz todas as alterações de estado até então.

4. Exemplos Práticos e Analogias

- **Analogia da Dockerização e Contêineres:** Ter um endereço de contrato fixo no frontend é como pintar a parede de uma casa alugada com uma cor muito específica. Se você precisar mudar de casa (de blockchain ou rede), terá que refazer todo o trabalho de pintura. Utilizar variáveis de ambiente é como usar luzes de LED programáveis na parede: você altera a cor projetada (o parâmetro do contrato) apenas mudando a configuração no painel (o contêiner), sem tocar na tinta (o código-fonte).
- **Analogia da Atomicidade (A Compra no Caixa):** Imagine que você vai ao supermercado. O

caixa registra as compras (passo 1: ler itens), você passa o cartão de crédito e o banco aprova o débito (passo 2: transferir dinheiro), mas a sacola plástica rasga e todos os produtos caem no chão (passo 3: erro inesperado). Na blockchain, a atomicidade garante que a transação inteira seja cancelada: os produtos voltam para a prateleira e o dinheiro é devolvido instantaneamente ao seu cartão, como se você nunca tivesse entrado na fila.

- **O Papel do `tx.wait()` (O Protocolo do Cartório):** Enviar uma transação para a blockchain é como dar entrada em uma escritura de imóvel no cartório. O atendente te dá um protocolo provisório de entrada (o objeto `tx`). Chamar `tx.wait()` equivale a aguardar o prazo legal para que o escrivão registre a escritura no livro oficial de imóveis (mineração no bloco). Apenas após esse carimbo oficial de confirmação você pode declarar que o imóvel é seu (exibir a confirmação na tela).

🎯 5. Resumo dos Pontos Críticos (O que vai cair na prova)

- A importância do desacoplamento de configurações e variáveis de ambiente em deploys profissionais (uso do `.env`).
- A diferença operacional no processamento de escrita: requer transmissão de transação, assinaturas digitais na MetaMask e pagamento de taxas de gas aos validadores.
- A utilização do método `tx.wait()` para coordenar a sincronização assíncrona entre a emissão da transação e sua mineração final no bloco de dados.
- A definição do princípio de **Atomicidade** em contratos inteligentes: execução integral ou reversão completa ao estado inicial.
- A sintaxe e comportamento do método `require` em Solidity como mecanismo central de validação e gatilho de reversão criptográfica na EVM.

? 6. Questões de Autoavaliação (Estilo Flashcard)

- **Pergunta 1: Explique por que o comando `await tx.wait()` é necessário no frontend após invocar uma função de escrita de um contrato inteligente.**

Ver Resposta Porque as transações que alteram dados na blockchain não são instantâneas. Quando a função de escrita é chamada, a carteira apenas assina e envia a transação para o pool de pendências da rede (mempool). O método retorna uma promessa com a transação pendente. O `tx.wait()` é necessário para travar a execução do código até que a transação seja de fato selecionada por um validador, processada e incluída em um bloco minerado, garantindo que a informação foi persistida na rede antes de o frontend prosseguir.

- **Pergunta 2: O que significa dizer que as transações executadas dentro da EVM (Ethereum Virtual Machine) possuem comportamento atômico?**

Ver Resposta Significa que a execução de todas as instruções presentes em uma transação funciona como uma unidade indivisível. Se todas as linhas de código da função forem executadas com sucesso, as alterações de estado são confirmadas globalmente na blockchain. Se ocorrer qualquer falha ou interrupção (como um estouro de gas ou validação incorreta no

``require``) no meio do caminho, toda a execução é abortada e a EVM restaura as variáveis de estado ao valor exato anterior ao início da transação.

- **Pergunta 3: Como a instrução `require` atua em Solidity e qual o efeito prático de sua falha no gas do usuário?**

Ver Resposta A instrução ``require`` valida uma pré-condição lógica obrigatória. Se a expressão for verdadeira, a execução prossegue normalmente. Se a expressão for avaliada como falsa, a execução do contrato é imediatamente interrompida, todas as alterações feitas no estado do contrato são revertidas (rolled back) e a transação é cancelada. O usuário perde a taxa de gas consumida até a linha do erro (que recompensa os validadores pelo processamento efetuado), mas o saldo principal da carteira e o estado do contrato são protegidos de alterações indevidas.

- **Pergunta 4: Para que servem expressões regulares (Regex) em aplicações frontend conectadas à Web 3.0? Dê um exemplo de padrão para validar inteiros.**

Ver Resposta Expressões regulares servem para validar dados de entrada no cliente (frontend) antes que a chamada de transação seja enviada à rede, evitando transações desnecessárias que falhariam na EVM (economizando gas). Um exemplo prático de padrão para validar que a string contém apenas números inteiros positivos é ``/^d+$/`` (onde ``^`` demarca o início, ``d+`` exige um ou mais dígitos numéricos, e ``$`` demarca o final da string).

- **Pergunta 5: Por que as alterações nas variáveis de estado de um contrato inteligente na rede local do Hardhat são perdidas quando o terminal do nó local é reiniciado?**

Ver Resposta Porque, por padrão, o nó local de simulação do Hardhat é executado inteiramente na memória RAM do computador de desenvolvimento. Quando o processo do terminal é finalizado, a memória volátil é limpa. Ao reiniciar o comando, o Hardhat recria a blockchain local a partir do bloco gênese (bloco zero), limpando o histórico de transações e contratos implantados anteriormente.

Aula 13

Disciplina: Blockchain e Web 3.0 **Vídeo de Referência:** @[aulas/videos/Aula 13 - Introdução à Web 3.0 (Parte 7).mp4]

✈ 1. Visão Geral e Objetivos

- **Contexto:** Esta aula aborda conceitos fundamentais da linguagem Solidity para contratos inteligentes, detalhando o tipo primitivo `address`, a estrutura interna de um contrato com sua própria carteira associada e a diferença entre transferências diretas e mediadas por lógica de código. Além disso, introduz os conceitos de custos operacionais (EVM opcodes), a alta precisão da rede com a unidade Wei, e a implementação prática de uma função segura para transferência de fundos.
- **Objetivos de Aprendizagem:** O aluno deve compreender a estrutura do tipo `address` e o checksum EIP-55, diferenciar transferências diretas e mediadas por contratos, entender a arquitetura de armazenamento global (*storage*) e custos de gas da EVM, explicar a relevância da unidade Wei para micropagamentos, e aplicar os modificadores `payable` e `external` na criação de contratos inteligentes.

🔑 2. Conceitos-Chave e Definições

- **`address`:** Tipo de dado nativo em Solidity que representa um identificador hexadecimal de 20 bytes (160 bits), servindo como a identidade de contas de usuário (EOAs) ou de contratos inteligentes implantados na blockchain.
- **EIP-55 (Ethereum Improvement Proposal 55):** Padrão que introduz um mecanismo de soma de verificação (*checksum*) para endereços Ethereum, utilizando capitalização sensível a maiúsculas e minúsculas (*case-sensitive*) para evitar transações incorretas por erros de digitação.
- **Transferência Direta (*Direct Transfer*):** Transação realizada diretamente entre duas contas de usuário (EOAs), sem a intermediação ou execução de códigos de um contrato inteligente (ex: envio manual via MetaMask).
- **Transferência Mediada (*Mediated Transfer*):** Transação financeira na qual o usuário interage com um contrato inteligente, que executa lógica pré-programada (validações, registro de eventos, dedução de taxas) antes de distribuir ou encaminhar os recursos.
- **Modificador `payable`:** Modificador de função ou endereço em Solidity que autoriza o recebimento de Ether na transação. Se uma chamada enviar valores financeiros para uma função que não seja `payable`, a EVM interromperá e reverterá o processo.
- **Modificador `external`:** Modificador de visibilidade que define que a função só pode ser chamada a partir de fora do contrato (por usuários ou outros contratos), otimizando o consumo de gas ao ler argumentos diretamente da `calldata` sem copiá-los para a memória RAM.
- **Endereço Zerado (*Zero Address* / `address(0)`):** Endereço com todos os 20 bytes compostos

por zeros ($0 \times 00 \dots 00$). É usado como destino para a queima (*burning*) irreversível de moedas ou como representação de um endereço inválido/inexistente.

- **Queima de Tokens (*Token Burning*)**: Processo de retirar tokens de circulação permanentemente enviando-os para um endereço sem chave privada conhecida (como o endereço zerado), atuando como um controle macroeconômico de escassez e inflação.
- **Espaço de Armazenamento (*Storage*)**: Memória global e persistente mantida por cada nó da blockchain para armazenar as variáveis de estado de um contrato inteligente. Escrever ou atualizar dados no *storage* é a operação computacional mais cara em gas.
- **Wei**: A menor fração divisível do Ether ($1 \text{ Ether} = 10^{18} \text{ Wei}$), cuja alta precisão permite a simulação de micropagamentos e cálculos financeiros de alta fidelidade sem o risco de erros de arredondamento comuns em float.
- **Opcodes (Códigos de Operação)**: Instruções computacionais básicas de baixo nível (Assembly) interpretadas pela Máquina Virtual do Ethereum (EVM). Cada operação (como *ADD*, *PUSH*, *MSTORE*, *CALL*) possui um custo fixo medido em unidades de gas.
- **Bytecode**: Cadeia binária de instruções em formato hexadecimal compilada a partir do Solidity, que é implantada diretamente nos blocos da blockchain e executada pela EVM.
- **Evento (*event*)**: Recurso usado em Solidity para registrar logs estruturados nos blocos da blockchain. Esses registros são mais econômicos em termos de gas do que a alteração de variáveis de *storage* e servem para notificar aplicações externas (frontend) em tempo real.

3. Explicação Detalhada por Tópicos

- **O Tipo *address* e Validação de Endereço (EIP-55)**:
 - O professor inicia destacando que o tipo *address* é a engrenagem essencial para qualquer contrato inteligente prático. Qualquer programa funcional manipula endereços de destino ou de origem.
 - O formato padrão é uma sequência de 40 caracteres hexadecimais (20 bytes).
 - O padrão **EIP-55** aplica uma validação de integridade (*checksum*) convertendo letras maiúsculas e minúsculas com base no hash do endereço. O navegador ou aplicação frontend realiza essa checagem antes de submeter a transação à rede, evitando o envio acidental de Ether para endereços sintaticamente corrompidos.
- **Arquitetura Interna de um Contrato Inteligente**:
 - Além do código de execução (*bytecode*) e das variáveis locais em memória, cada contrato inteligente possui:
 1. Seu próprio endereço exclusivo gerado no deploy.
 2. Seu próprio espaço de armazenamento persistente (*storage*).
 3. Sua própria carteira interna com saldo (*balance*) em Ether.
 - Essa estrutura permite que o contrato atue como um agente financeiro autônomo, acumulando saldo oriundo de serviços prestados ou distribuindo fundos sob condições programadas.

- **Modelos de Negócios Computacionais e Taxas (Analogia à Stripe):**

- É traçada uma comparação com a **Stripe**, empresa Web2 de processamento de pagamentos que cobra taxas de transação fixas e variáveis (ex: 4% do valor mais R\$ 0,39, ou 1,19% no Pix).
- Na Web3, desenvolvedores usam contratos inteligentes para programar intermediadores de pagamento. O próprio contrato pode reter automaticamente uma taxa operacional (ex: 1% de um item vendido em um jogo) direto em sua carteira interna antes de encaminhar o restante do valor ao vendedor, monetizando de forma puramente descentralizada.

- **Transferências Diretas vs. Transferências Mediadas por Contratos:**

- **Transferência Direta:** Transação simples em que o Ether sai da carteira A diretamente para a carteira B. O processo não envolve código de contrato e custa a taxa básica da blockchain (21.000 gas).
- **Transferência Mediada:** O usuário envia o valor para o contrato inteligente. O contrato analisa o contexto, executa lógicas customizadas (verifica banimentos, computa taxas, atualiza pontuações) e, se as regras forem satisfeitas, executa um repasse financeiro secundário para a conta de destino.

- **Queima de Moedas (*Burning*) e Controle Inflacionário:**

- No contexto descentralizado, o controle monetário da inflação ou escassez é feito via código.
- Enviar fundos para o **Zero Address** (`address(0)`) remove os tokens de circulação permanentemente, pois ninguém possui a chave privada desse endereço para resgatá-los.
- O professor discute como essa mecânica simula políticas monetárias de um Ministério da Economia para equilibrar o poder de compra e o valor de um ativo no mercado.

- **Listas de Permissões e o Alto Custo de Storage:**

- Ao planejar uma lista de endereços banidos ou autorizados (ex: `address[] private invalidAddresses`), o desenvolvedor deve se atentar ao custo de gas.
- O armazenamento permanente na blockchain (*storage*) é escasso e custoso. Cada bloco tem um limite computacional de processamento.
- Salvar uma transação estruturada na blockchain global (contendo carteira de origem, destino, valor e data/hora) consome cerca de 128 bytes por bloco. Escrever esses dados repetidamente eleva drasticamente a taxa paga pelos usuários.

- **Frações Monetárias (Wei) e Micropagamentos:**

- Diferente do sistema financeiro fiduciário limitado a duas casas decimais, o Ether possui 18 casas decimais. A menor unidade é o **Wei** (10^{-18} Ether).
- Essa granularidade extrema viabiliza micropagamentos em escala global (ex: cobrar 100 Wei por caractere de texto traduzido). Em milhões de interações, o saldo se acumula de forma precisa sem perdas por arredondamento.

- **Compilação, Opcodes da EVM e o Cálculo do Gas:**

- O código em Solidity é compilado em *bytecode* hexadecimal compreendido pela EVM.
- O bytecode traduz as linhas em instruções computacionais de baixo nível chamadas **opcodes** (como `PUSH`, `MSTORE`, `ADD`, `CALL`).
- Cada opcode consome uma quantidade tabelada de gas (ex: operação de adição consome 3 gas). O valor cobrado na MetaMask do usuário reflete exatamente a soma dos custos de gas de todos os opcodes executados pela transação na EVM.

- **A Estrutura da Função Segura de Transferência (`transferEther`):**

- O professor exemplifica a lógica de transferência segura em Solidity:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract EtherSender {
    // Declaração do evento de auditoria
    event TransferExecuted(address indexed from, address indexed to, uint256 value);

    // Função externa habilitada para receber Ether (payable)
    function transferEther(address payable to) external payable {
        // 1. Garante que o destino não é o endereço de queima (zero address)
        require(to != address(0), "Endereco de destino invalido");

        // 2. Garante que o usuário enviou algum valor na chamada (msg.value em Wei)
        require(msg.value > 0, "O valor enviado deve ser maior que zero");

        // 3. Executa a transferência de baixo nível com tratamento de retorno
        (bool success, ) = to.call{value: msg.value}("");
        require(success, "Falha na transferencia");

        // 4. Emite o evento registrando a transação com baixo consumo de gas
        emit TransferExecuted(msg.sender, to, msg.value);
    }
}
```

- O uso do método `call` com envio de gas flexível e retorno booleano é a prática moderna recomendada pelo Solidity para enviar fundos, mitigando vulnerabilidades. A instrução `require(success)` assegura que se a transferência de rede falhar, a transação inteira será revertida, protegendo os fundos.

4. Exemplos Práticos e Analogias

- **Analogia da Stripe (Web2) vs. Contratos Inteligentes (Web3):** Imagine uma loja virtual. Na Web2, quando um cliente compra um produto de R\$ 100,00, a API da Stripe processa o pagamento, cobra uma taxa de R\$ 4,39 e repassa R\$ 95,61 para a conta bancária da loja dias depois. Na Web3, um contrato inteligente `payable` assume esse papel sem depender de uma

corporação intermediária: o cliente faz a compra, o contrato retém 1% (R\$ 1,00) de taxa de serviço para a carteira de taxas do contrato e repassa automaticamente R\$ 99,00 para a carteira do vendedor na blockchain de forma instantânea e transparente.

- **Analogia da Trituradora de Dinheiro (Token Burning):** Pense em uma economia nacional onde o Banco Central recolhe milhões de cédulas de papel-moeda de circulação e as tritura fisicamente sem repor o estoque circulante. A redução da oferta monetária torna o dinheiro restante em circulação teoricamente mais escasso e valorizado. Enviar tokens ou Ether para o `address(0)` funciona exatamente como essa trituradora de papel: o dinheiro digital entra em um endereço público inalterável e inacessível por qualquer chave privada, extinguindo as moedas de circulação do mercado.
- **A Divisão de Letras (A Precisão de 18 Casas do Wei):** Tentar cobrar frações de centavos por caractere digitado em um editor de texto utilizando cartões de crédito tradicionais é impossível porque a menor unidade monetária é o centavo (\$0,01\$) e as taxas fixas bancárias inviabilizam micropagamentos. Na blockchain, a alta precisão do Wei possibilita cobrar 100 Wei (\$0,000000000000000001\$ Ether) por caractere traduzido. O usuário pode traduzir 500 caracteres, pagar exatos 50.000 Wei (uma fração minúscula de centavo) e ter sua transação computada e validada sem perdas decimais.

5. Resumo dos Pontos Críticos (O que vai cair na prova)

- **Finalidade do modificador `payable`:** Exigido para autorizar funções e variáveis do tipo `address` a receberem ou transferirem valores financeiros em Ether.
- **Vantagens do modificador `external`:** Indica à EVM que a função não será invocada internamente pelo próprio contrato. Isso reduz taxas de gas pois os dados de entrada permanecem intocados na `calldata`, sem alocação ou cópia na memória RAM.
- **Segurança contra o `address(0)`:** Validação mandatória no início de qualquer fluxo de transferência de fundos para impedir que o Ether seja perdido permanentemente no endereço zerado de queima.
- **Opcodes e Custo Computacional de Gas:** A taxa de rede é a soma dos custos de gas tabelados para cada opcode (instrução assembly de baixo nível) executado pela EVM. Operações que alteram dados permanentes de *storage* são significativamente mais caras do que operações de leitura ou gravação de logs de eventos.
- **Uso de `call` para Envio de Ether:** O método `to.call{value: msg.value}("")` é a instrução padrão da linguagem para enviar fundos, pois transfere todo o gas restante para o receptor e retorna um booleano de sucesso que deve ser verificado com `require`.
- **Relação de Escala de Wei:** Saber converter e compreender que $1 \text{ Ether} = 10^{18} \text{ Wei}$, a unidade fundamental de cálculo interno da EVM.

? 6. Questões de Autoavaliação (Estilo Flashcard)

- **Pergunta 1:** Por que é necessário marcar um argumento ou variável de endereço como `address payable` em vez de apenas `address` quando pretendemos transferir Ether a ele?

Ver Resposta Em Solidity, o tipo básico `address` não permite o recebimento direto de transferências financeiras. Para habilitar os métodos de envio de Ether (como `.transfer()`, `.send()` ou `.call{value: ...}()`), o tipo de endereço deve ser explicitamente declarado ou convertido para `address payable`. Trata-se de uma proteção de segurança do próprio compilador para evitar o envio de dinheiro a endereços que não estão preparados para recebê-lo.

- **Pergunta 2: Explique a diferença no consumo de gas entre ler/escrever variáveis locais no escopo de uma transação e gravar permanentemente informações no *storage* do contrato.**

Ver Resposta Escrever dados no *storage* (armazenamento persistente global da blockchain) é a operação mais cara em termos de gas na EVM porque exige que todos os nós da rede registrem e guardem aquela informação em seu histórico permanente físico de disco. Variáveis locais (declaradas no escopo de funções como `memory` ou `calldata`) duram apenas durante o processamento da transação e são apagadas da RAM logo após o término, custando apenas uma pequena fração do gas cobrado pelo storage.

- **Pergunta 3: Qual a diferença prática entre uma transferência direta de Ether e uma transferência mediada por contrato inteligente?**

Ver Resposta A transferência direta é um envio de fundos de uma carteira de usuário (EOA) para outro endereço sem envolver código de programação, gerando apenas um registro no bloco com a taxa básica de gas (21.000). A transferência mediada direciona os fundos para o endereço de um contrato inteligente, que executa lógica e códigos personalizados (como auditorias de segurança, dedução de taxas de protocolo, validação de regras de negócios) antes de reencaminhar o valor ao destino definitivo.

- **Pergunta 4: O que são os Opcodes da EVM e como eles influenciam o valor da taxa de gas que o usuário confirma na carteira MetaMask?**

Ver Resposta Opcodes são códigos de operação de baixo nível (Assembly) resultantes da compilação do Solidity que a Máquina Virtual do Ethereum executa (ex: `PUSH`, `MSTORE`, `ADD`). Cada opcode tem um custo pré-determinado de unidades de gas na especificação da rede. O custo de gas exibido e pago na MetaMask é a soma exata de todos os opcodes executados pela transação computada no nó validador multiplicada pelo preço do gas no momento da rede.

- **Pergunta 5: Por que é considerado uma boa prática de segurança adicionar a linha `require(to != address(0), "...")` antes de efetuar transferências em um contrato Solidity?**

Ver Resposta Porque o `address(0)` (`0x00...00`) é um endereço especial do sistema que não tem chave privada correspondente e atua como uma conta de queima. Se fundos forem enviados para esse endereço por engano ou bug no frontend, eles serão perdidos para sempre. O `require` intercepta a transação antes do envio, gerando um erro de reversão de estado que

devolve o Ether ao usuário de origem e evita o extravio definitivo do dinheiro.

Aula 14

Disciplina: Blockchain e Web 3.0 **Vídeo de Referência:** @[aulas/videos/Aula 14 - Introdução à Web 3.0 (Parte 8).mp4]

✈ 1. Visão Geral e Objetivos

- **Contexto:** Esta aula conclui o ciclo prático introdutório da disciplina e avança para a integração de interfaces web com a blockchain usando a biblioteca `ethers.js`. O foco central é compreender a passagem de valores monetários a funções `payable`, a manipulação de metadados de transação via objeto `msg`, a emissão de logs através de eventos e a depuração de contratos localmente por meio de um explorador de blocos.
- **Objetivos de Aprendizagem:** O aluno deve compreender o papel e o preenchimento das propriedades `msg.sender` e `msg.value` pela EVM, saber codificar chamadas de funções `payable` enviando Ether convertido para Wei via frontend, declarar e emitir eventos em Solidity capturando-os no JavaScript, e instalar e configurar um explorador de blocos local conectado à Testnet do Hardhat.

🔑 2. Conceitos-Chave e Definições

- **Objeto `msg`:** Objeto global implícito injetado automaticamente pela Máquina Virtual do Ethereum (EVM) em todas as funções executadas, carregando metadados da transação em andamento.
- **`msg.value`:** Propriedade de leitura do objeto `msg` que informa a quantidade exata de Ether (expressa na unidade inteira Wei) enviada pelo remetente da transação junto à chamada da função.
- **`msg.sender`:** Propriedade de leitura do objeto `msg` que armazena o endereço da conta (carteira de usuário ou contrato) que assinou e efetuou a chamada atual da função.
- **Evento (`event`):** Estrutura em Solidity que registra de forma permanente informações estruturadas em logs de transação armazenados nos blocos da rede, sem alterar variáveis globais do contrato.
- **Emissão de Evento (`emit`):** Comando em Solidity responsável por disparar e consolidar os dados de um evento declarado nos logs históricos da blockchain durante o processamento de uma função.
- **`tx.wait()`:** Função assíncrona pertencente à biblioteca `ethers.js` que suspende temporariamente a linha de execução do frontend até que o bloco contendo a transação pendente seja minerado e confirmado pelos validadores.
- **Explorador de Blocos Local:** Aplicação visual instalada localmente (geralmente baseada em React) que exibe blocos, mempool, transações pendentes, opcodes da EVM e o consumo de gas associado a uma rede Hardhat local de testes.
- **Conversão Ether-Wei (`parseEther`):** Operação matemática utilitária do frontend para converter representações decimais de moedas (ex: 2.5 Ether) em inteiros em escala Wei (10^{18}),

formato obrigatório para processamento interno da rede.

- **Hash de Transação (*Transaction Hash*)**: Identificador hexadecimal único de 32 bytes gerado criptograficamente para rastrear, validar e referenciar cada transação enviada e registrada na blockchain.
- **Taxa de Protocolo / Intermediação (*Protocol Fee*)**: Fração financeira deduzida do valor principal transacionado em uma função que fica retida permanentemente no saldo do contrato intermediador como taxa de serviço, permitindo a monetização do protocolo Web3.

3. Explicação Detalhada por Tópicos

- **Fechamento do Ciclo HelloWorld:**

- O professor consolida as bases teóricas do primeiro contrato simples de escrita e leitura de dados:
 - Variáveis globais em Solidity residem no **storage**, a memória de estado mais duradoura e custosa de cada nó validador.
 - Todo contrato inteligente implantado possui um endereço próprio e comporta saldo de forma semelhante a uma carteira pessoal de usuário (EOA).
 - Modificadores **view** e **pure** indicam funções que apenas leem dados ou não acessam nenhuma memória, respectivamente, poupando a execução de transações on-chain complexas.

- **O Objeto **msg** Injetado pela EVM:**

- A EVM injeta implicitamente o objeto global **msg** em todas as funções chamadas, sem necessidade de declaração de parâmetros locais.
- O atributo **msg.sender** identifica de forma segura qual chave privada assinou e executou a chamada (essencial para controle de acesso).
- O atributo **msg.value** carrega o valor exato de fundos enviados para a execução da lógica de negócios.

- **Integração de Escrita com Envio de Ether (**ethers.js**):**

- O input do usuário em representação decimal (ex: 10 Ether) é convertido pelo JavaScript usando **ethers.utils.parseEther("10")** para gerar a representação inteira em Wei.
- Para chamar funções **payable** enviando Ether, o frontend passa os argumentos regulares do método Solidity e adiciona ao final um objeto de configuração com a propriedade **value**:

```
// Código Javascript (Frontend)
const tx = await contrato.transferEther(enderecoDestino, {
  value: ethers.utils.parseEther("10.0") // Envia 10 Ether em Wei
});
```

- O MetaMask intercepta o objeto de configuração, abre a tela de consentimento exibindo o valor de 10 Ether a ser retirado do saldo do remetente e a respectiva taxa de gas da rede.

- **Sincronismo de Rede e Mineração via `tx.wait()`:**

- No frontend, a execução do comando `contrato.transferEther(...)` retorna imediatamente uma promessa com a transação pendente (`tx`), contendo apenas o seu hash identificador na mempool.
- Chamar `await tx.wait()` trava o processamento do JavaScript até que a transação saia da fila e seja de fato selecionada por um nó, incluída em um bloco de dados e minerada.
- Somente após a resolução dessa promessa é que se atualiza a interface para o status de "Transferência Concluída".

- **Auditoria de Logs com Eventos em Solidity:**

- Manter registros históricos estruturados no storage do contrato inteligente é proibitivo pelo alto consumo de gas de rede.
- A melhor prática é a declaração e disparo de **eventos** (`event`). Os eventos criam registros nos logs do bloco de forma econômica e podem ser monitorados e indexados por sistemas de indexação externos (como subgraphs ou webhooks).
- Estruturação e disparo de evento no contrato:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract EtherSender {
    // Declaração do evento
    event TransferExecuted(address indexed from, address indexed to, uint256 value);

    function transferEther(address payable to) external payable {
        require(to != address(0), "Destinatario invalido");
        require(msg.value > 0, "O valor precisa ser maior que zero");

        (bool success,) = to.call{value: msg.value}("");
        require(success, "Falha na transferencia");

        // Emissão do evento
        emit TransferExecuted(msg.sender, to, msg.value);
    }
}
```

- O marcador `indexed` permite filtrar eventos na blockchain baseados nos endereços de origem (`from`) ou destino (`to`).
- O frontend JavaScript pode acessar esses logs através do array retornado no recibo da transação (`receipt.events`).

- **O Explorador de Blocos Local:**

- O professor apresenta o uso de um explorador local (como o Blockscout adaptado ou clone visual de Etherscan) para auditar execuções da rede de testes local (Hardhat).

- **Passos para Instalar e Executar o Explorador:**

1. Executar o clone do repositório do explorador no diretório local.
2. Instalar todas as dependências locais utilizando o comando `npm install`.
3. Copiar o arquivo `.env.example` para criar o arquivo `.env`.
4. Atualizar o arquivo `.env` especificando o endpoint RPC da rede local do Hardhat (ex: `REACT_APP_RPC_URL=http://127.0.0.1:8545`).
5. Rodar a aplicação utilizando `npm start`.

- **Lógica de Retenção de Taxas Operacionais (Evolução de Negócios):**

- O professor introduz a teoria das taxas de protocolo (Stripe Model).
- Em vez de simplesmente encaminhar 100% de `msg.value` para a conta de destino, o contrato aplicará uma fórmula matemática para deduzir uma taxa operacional (ex: 5%):
$$\text{Taxa} = \frac{\text{msg.value} \times 5}{100}$$

$$\text{Valor Líquido} = \text{msg.value} - \text{Taxa}$$
- O contrato executa a transferência apenas do **Valor Líquido** para a carteira final, e o restante (os 5% de taxa cobrada) é retido de forma automática no balanço da carteira vinculada ao próprio contrato inteligente.

4. Exemplos Práticos e Analogias

- **Analogia da Carta Registrada (O Objeto `msg`):** Imagine que você envia uma carta de remessa de valores a um banco contendo cédulas dentro do envelope. Você não precisa escrever uma nota interna informando qual é o seu nome e quanto colocou lá dentro para que o caixa saiba o remetente. O próprio envelope físico selado traz o remetente oficial carimbado no verso e o valor é aferido pelo peso e verificação de recebimento. Em Solidity, a EVM trata o objeto implícito `msg` exatamente como esse envelope carimbado: ele insere de forma inviolável quem assinou (`msg.sender`) e quanto dinheiro foi anexado à transação (`msg.value`).
- **Analogia da Senha do Painel (O `tx.wait()`):** Entregar uma transação à blockchain é como dar entrada em um requerimento no guichê de um cartório. O funcionário aceita os papéis, carimba e te entrega uma ficha de protocolo com um código de barras de rastreio (o hash da transação `tx`). Você não sai com o processo julgado nesse momento. Você precisa se sentar e aguardar que o painel luminoso chame a sua senha para retirar o documento finalizado (a promessa assíncrona do `tx.wait()`). Somente após o painel eletrônico confirmar o arquivamento (mineração) é que o trâmite está oficialmente ativo.
- **Analogia da Voz do Professor vs. Anotação na Lousa (Eventos vs. Storage):** Se o professor quisesse registrar a presença histórica de alunos durante 10 anos escrevendo o nome de cada um na lousa física da sala e deixando-os desenhados lá para sempre, a lousa ficaria sem espaço útil rapidamente, exigindo a compra de novas salas e paredes de concreto (alto custo de *storage* global). Em vez disso, o professor apenas diz em voz alta "Aluno X está presente" (emite o evento). Um fiscal do corredor anota isso em um livro de presença externa

em papel (logs da transação). A lousa permanece livre para operações do dia a dia a um custo financeiro irrisório para a escola.

🔗 5. Resumo dos Pontos Críticos (O que vai cair na prova)

- **Preenchimento Automático do Objeto `msg`:** Reconhecer que propriedades como `msg.sender` (remetente) e `msg.value` (valor em Wei) são injetadas nativamente pela EVM a partir da assinatura criptográfica e do payload do remetente.
- **Configuração de Chamadas Payable no Ethers.js:** Saber que o Ether enviado para funções `payable` é especificado via frontend em um parâmetro extra do tipo objeto `{ value: valorEmWei }` passado no final da chamada de função.
- **Coordenação Assíncrona via `tx.wait()`:** Compreender que a confirmação no frontend de uma escrita depende do método `wait()` para suspender o JavaScript até a finalização da mineração no bloco da rede.
- **logs de Transações via Eventos:** Compreender a diferença econômica e arquitetural dos eventos (`event` e `emit`) em relação a arrays em storage global para fins de gravação de dados históricos.
- **Retenção de Saldos de Taxas no Contrato:** Entender que a retenção do excedente financeiro de uma transação dentro da carteira interna do contrato ocorre automaticamente ao não repassar 100% do valor de `msg.value` na chamada de transferência de baixo nível.

? 6. Questões de Autoavaliação (Estilo Flashcard)

- **Pergunta 1: Como informamos à biblioteca `ethers.js` que queremos enviar Ether de nossa carteira para uma função payable de um smart contract? Dê um exemplo prático da linha de chamada.**

Ver Resposta Para enviar Ether em uma chamada, passamos um objeto JavaScript como último parâmetro da chamada da função no frontend contendo a propriedade ``value`` com a quantidade convertida para Wei. Exemplo: ``await contrato.transferEther(destino, { value: ethers.utils.parseEther("5") });``

- **Pergunta 2: O que representam as propriedades `msg.sender` e `msg.value` em Solidity e como elas ajudam na segurança e na lógica financeira dos contratos?**

Ver Resposta ``msg.sender`` armazena o endereço público da conta que realizou e assinou criptograficamente a chamada, impedindo fraudes de identidade. ``msg.value`` armazena o valor exato em Wei enviado juntamente com a chamada de função payable, o que permite ao contrato auditar se a quantidade de Ether recebida satisfaz os requisitos financeiros necessários antes de prosseguir com a execução.

- **Pergunta 3: Explique a diferença de custo e persistência entre emitir dados por meio de um `event` e salvá-los no array de um contrato inteligente.**

Ver Resposta Emitir dados por eventos grava as informações diretamente na seção de logs

das transações do bloco na blockchain. Esse método consome significativamente menos gas porque logs não fazem parte do estado global mutável do contrato e não precisam estar disponíveis para acesso direto em tempo de execução por outros smart contracts. Salvar dados no storage em arrays exige modificação de variáveis globais persistentes na memória de estado dos nós da blockchain, demandando muito mais poder computacional e tarifas elevadas de gas fee.

- **Pergunta 4: O que faz o método `tx.wait()` no JavaScript do frontend e o que ocorreria se a interface do usuário tentasse exibir o saldo do contrato sem aguardar a conclusão dessa chamada?**

Ver Resposta O `tx.wait()` bloqueia de forma assíncrona o fluxo de execução até que a transação correspondente saia da mempool (pool de transações pendentes), seja processada e minerada formalmente em um novo bloco. Se o frontend tentasse carregar e exibir o saldo do contrato antes de aguardar o término do `wait()`, a interface exibiria o saldo antigo desatualizado, pois a transação ainda estaria sendo propagada pela rede.

- **Pergunta 5: Como um contrato inteligente consegue acumular fundos em sua carteira própria baseando-se no modelo de intermediação de pagamentos com taxa de serviço (Stripe Model)?**

Ver Resposta Quando o usuário invoca a função `payable` enviando fundos, todo o valor especificado em `msg.value` entra para a custódia temporária do endereço do contrato. A lógica do contrato realiza a dedução matemática de uma taxa (ex: 5% do valor recebido) e transfere apenas o valor restante (95%) para o destinatário final. Como o contrato executa o comando `.call{value: 95%}(...)`, a diferença (os 5% de taxa) não é repassada e permanece depositada permanentemente no balanço financeiro da própria carteira do contrato.